



Natural language-driven production planning: integrating large language models with automatic simulation model generation in manufacturing systems

Mohaiad Elbasheer¹ · Yuanjun Laili² · Francesco Longo¹ · Vittorio Solina¹ · Yiran Tao² · Pierpaolo Veltri^{1,2} · Yuteng Zhang² · Lin Zhang²

Received: 14 June 2025 / Accepted: 28 October 2025
© The Author(s) 2025

Abstract

Simulation modeling provides powerful decision support capabilities for production planning in manufacturing systems, yet widespread adoption remains limited by significant barriers in model development and maintenance. Traditional simulation approaches require substantial expertise, time investment, and financial resources, creating accessibility challenges particularly for small and medium enterprises. Current research on automatic simulation generation overlooks the significant potential that generative AI can offer in bridging the gap between natural language expression and executable model creation. This paper presents a novel methodology integrating large language models (LLMs) with automatic simulation model generation (ASMG) to enable direct generation of executable simulation models from natural language conversations. Unlike existing ASMG approaches that require formal specifications or simulation knowledge, our framework reduces programming requirements by allowing users to create functional models through conversational interaction. Our approach leverages natural language processing capabilities to transform user requests into executable simulation models without requiring specialized programming knowledge, enhancing human–machine interaction in production planning environments. The framework implements a four-step process: user intent expression through natural language interfaces, knowledge extraction using structured templates, simulation model construction via object-oriented data-driven components, and simulation execution with comprehensive result analysis for resource utilization, bottleneck identification, and production scheduling optimization. Performance testing demonstrates successful model generation across 18 supported action types with response times ranging from 8 s for simple operations to 5 min for complete manufacturing system creation. By enabling direct conversion of natural language requests into functional simulation models, this LLM-enhanced ASMG approach represents a step toward expanding access to simulation technology for non-expert users while maintaining the analytical power needed for complex manufacturing decision support and production control.

Keywords Simulation · Production planning · Decision support · GenAI · LLMs

Mohaiad Elbasheer, Yuanjun Laili, Francesco Longo, Vittorio Solina, Yiran Tao, Pierpaolo Veltri, Yuteng Zhang, and Lin Zhang contributed equally to this work.

✉ Pierpaolo Veltri
pierpaolo.veltri@unical.it

¹ Department of Mechanical, Energy and Management Engineering, University of Calabria, Rende, Italy

² School of Automation Science and Electrical Engineering, Beihang University, Beijing, China

Introduction

Simulation models serve as indispensable decision support tools across numerous industrial sectors, enabling stakeholders to test scenarios, optimize processes, and predict outcomes without disrupting operational systems. These virtual representations provide critical insights for intelligent manufacturing facilities, production systems, supply chains, healthcare systems, and service operations by capturing complex system behaviors and interactions (Katsaliaki & Mustafee, 2019; Negri et al., 2021; Steinbacher et al., 2023b). Despite their proven value, widespread adoption of

simulation technology remains constrained by significant barriers (Collins et al., 2023). Developing effective simulation models typically requires months of time-consuming effort from specialized experts with advanced knowledge of both simulation methodologies (Soufi et al., 2024). This expertise gap creates dependency on a limited pool of simulation professionals, resulting in bottlenecks and substantial costs ranging from tens to hundreds of thousands of dollars for complex industrial models (Hill, 2022). Furthermore, the rapid evolution of modern production environments means models quickly become outdated, necessitating continuous maintenance that compounds time, expertise, and financial demands (Alieva & Haartman, 2020; Soares et al., 2023). These combined factors place simulation technology beyond the reach of many potential beneficiaries, particularly small and medium enterprises (SMEs) (Ayandibu & Houghton, 2017; Ghobakhloo & Ching, 2019).

This accessibility gap has driven extensive research into Automatic simulation model generation (ASMG) methodologies that aim to streamline and partially automate the model development process. Indeed, while these efforts have yielded valuable advances through data-driven, algorithmic, knowledge-based, and interface-facilitated approaches (Hughes et al., 2013; Reinhardt et al., 2019; Schlecht et al., 2023), most existing solutions still require users to possess foundational knowledge of simulation concepts and formal specification languages. In response to this persistent challenge, the emergence of large language models (LLMs) represents a potential paradigm shift for addressing this limitation. Specifically, these models demonstrate notable capabilities in understanding natural language, generating code, adapting to technical domains, and providing explanatory rationales for their outputs (Chang et al., 2024; Hadi et al., 2023; Jiang et al., 2024; Patil & Gudivada, 2024; Wang et al., 2025). Furthermore, recent work by Zhang et al. (2025) has shown promising results in integrating LLMs with simulation model generation in model-based systems engineering contexts, demonstrating the feasibility of this approach within the product engineering domain. Nevertheless, Behrendt et al. (2025) and Cimino et al. (2025) specifically highlight the lack of LLM integration in existing ASMG methodologies as a limitation of current approaches. Consequently, the vision of truly accessible simulation technology, where domain experts without simulation backgrounds could express modeling needs in familiar language, has remained largely unfulfilled, particularly in human–machine interaction through natural language interfaces (Son et al., 2003).

This research addresses the following questions:

RQ1: How can natural language interfaces enable non-expert users to create simulation models for production planning?

RQ2: What are the performance characteristics of LLM-enhanced simulation model generation across different operation types

To address these challenges, this paper presents a novel methodology that integrates large language models with automatic simulation model generation for production planning applications. The main contributions of this research include:

1. Development of a natural language interface that enables users to generate simulation models for production planning without specialized programming knowledge or simulation expertise.
2. Integration of LLMs with ASMG through a four-phase framework that combines natural language processing with structured knowledge extraction, addressing common LLM limitations while handling varied user input specifications in manufacturing contexts.
3. Performance evaluation demonstrating the framework's response times and operational capabilities across different action types, providing baseline metrics for LLM-enhanced simulation model generation.

The approach aims to reduce barriers to simulation technology adoption by enabling model generation through conversational interaction while maintaining computational accuracy for manufacturing decision support.

The paper is organized as follows: In “[State-of-the-art overview](#)” section reviews current ASMG methodologies and examines the potential for LLM integration. In “[Methodology](#)” section presents the four-phase methodology and technical implementation. In “[Results](#)” section evaluates the framework through practical application, performance testing, and baseline comparison with traditional ASMG approaches across multiple manufacturing domains. In “[Discussion](#)” section discusses findings, limitations, and future directions, with conclusions in “[Conclusion](#)” section.

State-of-the-art overview

Simulation modeling has become essential for industrial decision support, though widespread adoption remains limited by significant development barriers. This section reviews current approaches and identifies opportunities for enhancing accessibility through language-based interfaces.

Simulation and current challenges

Simulation models serve as crucial enabling technologies for decision support across industrial applications, with a particular emphasis on production planning, along with

supply chain management, process optimization and predictive maintenance (Katsaliaki & Mustafee, 2019; Santos et al., 2022; Steinbacher et al., 2023b; Zonta et al., 2020). In manufacturing contexts, stakeholders leverage these models to evaluate the impact of changes in production lines, resource allocation, and cycle times, enabling the identification of bottlenecks and improvement of overall efficiency (Ferreira et al., 2020; Sturrock, 2019; Tiago et al., 2020). The versatility of simulation technologies extends throughout Industry 4.0 and its evolution into the 5.0 paradigm (Longo et al., 2023; Zhang et al., 2019). In production planning specifically, simulation provides powerful capabilities for optimizing schedules, resource allocation, and workflow sequencing (Ferreira et al., 2020; Negri et al., 2021; Steinbacher et al., 2023b; Wocker et al., 2023). These digital models enable production managers to test various planning scenarios, identify constraints, and develop robust schedules that maximize throughput while minimizing costs (Cimino et al., 2024; Tiago et al., 2020). Beyond planning, these digital twins and virtual environments offer manufacturers the ability to test scenarios without real-world risks, providing a foundation for more informed strategic decisions (Orlova, 2022; Wocker et al., 2023). Simulation facilitates virtual prototyping, allowing product designs to be tested and modified in digital space before physical production, significantly reducing development costs and time-to-market (Uhlmann & Frazzon, 2018). Moreover, predictive simulation models enable organizations to forecast equipment failures, schedule preventive maintenance interventions, and reduce unplanned downtime while extending machine service life (Geurtsen et al., 2023; Zonta et al., 2020). In the supply chain context, simulation technology supports the design and optimization of logistics networks (Cadi et al., 2021; Sarraj et al., 2014), enhancing resilience against disruptions and improving overall operational performance. Additionally, these technologies provide safe and controlled environments for employee training, allowing industrial operators to gain skills and familiarize themselves with new processes without exposure to operational risks (Longo et al., 2023).

Despite these benefits, developing simulation models requires high-level skills from simulation experts, substantial time investments, and significant financial resources (Soufi et al., 2024). A major limitation of simulation in industrial applications is its effectiveness over time. As noted by Soares et al. (2023), modern industry is characterized by rapidly evolving production processes that respond to market needs, causing simulation models to struggle with persistent changes and often becoming obsolete shortly after implementation. The time-consuming nature of traditional simulation approaches compounds this problem, as

models frequently require extensive redevelopment rather than simple updates. This continuous need for updating and maintenance leads to what Alieva and Haartman (2020) term “digital waste”, applications that are no longer usable due to changes in system requirements and parameters. Furthermore, accessibility remains a significant barrier, particularly for small and medium enterprises (SMEs) that face constraints such as high costs, technical demands, and limited resources. This creates what Ghobakhloo and Ching (2019) describes as a “selective industrial revolution,” where only large companies with sufficient resources can fully benefit from Industry 4.0 technologies, limiting opportunities for innovation and growth across the broader industrial landscape (Ayandibu & Houghton, 2017).

These challenges reveal a common limitation across traditional simulation approaches: they all require significant human expertise at critical points like initial model development, ongoing maintenance, or adaptation to new requirements. ASMG methodologies have emerged as potential solutions to streamline and partially automate the model development process, addressing these persistent barriers through various technological approaches.

Evolution of automatic simulation model generation

The persistent challenges identified in simulation adoption, including model obsolescence due to rapidly evolving industrial environments, dependency on specialized expertise, and limited accessibility particularly for SMEs, have driven extensive research into methodological solutions that can reduce development barriers while maintaining analytical capabilities. To address these challenges, intensive efforts have focused on creating flexible simulation-based systems and applications. ASMG has emerged as a methodological framework that allows for the seamless creation and adaptation of models to reflect changes in real-world systems (Cimino et al., 2025; Hughes et al., 2013; Reinhardt et al., 2019; Schlecht et al., 2023). This approach facilitates the construction of models inherently capable of adjusting to new operational conditions automatically, marking a significant step toward reducing the time and resources traditionally required for simulation model development and deployment (Harrison et al., 2007; Krenczyk et al., 2018). ASMG development has evolved through four distinct but complementary perspectives, each addressing different aspects of the model generation challenge (Cimino et al., 2025):

1. *Data-centric approaches* Data-centric approaches leverage empirical data from physical systems to inform

model development. These function through event-centric process mining for automated discovery of manufacturing systems (Castiglione, 2024), data-driven frameworks using machine learning for semiconductor equipment emulation (Behrendt et al., 2025), and structured data catalogs for material handling systems (Soufi et al., 2024). Primary strategies include data-driven modeling, parameterization, and model reduction techniques that maintain essential dynamics while simplifying computation (Friederich et al., 2022; Lugaresi & Matta, 2021).

2. *Algorithmic approaches* Algorithmic approaches employ computational intelligence to streamline model construction. These work through process mining techniques that extract production system behavior from trace data (Maier et al., 2024), machine learning applications for predicting equipment behavior (Garcia et al., 2023), and integration of neural networks for object detection and dynamic reconfiguration (Mo et al., 2023; Sommer et al., 2020). Core methods include machine learning for complex behavior emulation, optimization techniques for efficient configurations, and rule-based modeling to capture system logic (Mo et al., 2023; Tan et al., 2023).
3. *Knowledge-based approaches* Knowledge-based approaches formalize model development using domain expertise and structured representations. These operate through template-based approaches combined with structural modeling (Elbasheer et al., 2024), component-based synthesis for automatic generation of structural variants (Winkels et al., 2024), and comprehensive ontologies for converting planning data into simulation models (Göppert et al., 2023; Steinbacher et al., 2023a). Implementations demonstrate flexibility through object-oriented modular architectures that adapt to physical system changes without manual intervention (Cimino et al., 2024). Main strategies include templates as customizable blueprints, structural frameworks, ontology-based knowledge representations, and model reuse within new contexts (Billiet & Stark, 2022; Kaiser et al., 2022).
4. *Interface-facilitated approaches* Interface-facilitated approaches enhance user interaction with simulation systems for intuitive model creation. This perspective emphasizes the role of user interfaces in model generation, enabling easier simulation development (Leißau and Laroque (2024). The approach includes graphical user interfaces for visual interaction, natural language interfaces for everyday language interpretation, and interactive dialogue interfaces for conversational model development (Son et al., 2003; Steinbacher et al.,

2023a). The implementation of natural and interactive dialogue interfaces is still nascent but shows potential for revolutionizing simulation model development in industrial settings, according to Cimino et al. (2025).

While all four ASMG perspectives address different aspects of simulation development, they vary significantly in their accessibility to non-expert users. Data-centric and algorithmic approaches, despite offering high automation capabilities, require substantial technical expertise for implementation and maintenance, limiting their accessibility. Knowledge-based approaches provide effective solutions for domain experts through templates and structural frameworks, achieving successful industrial deployment but maintaining barriers for non-specialists. Interface-facilitated approaches emerge as the most promising perspective for democratizing simulation technology, prioritizing user accessibility through intuitive graphical interfaces and natural language capabilities that enable non-expert users to create simulation models without specialized technical knowledge. This enhanced usability positions interface-facilitated approaches as the key enabler for widespread simulation adoption across diverse industrial contexts, addressing the fundamental accessibility gap that has historically limited simulation technology to specialized users. However, it must be explicitly recognized that none of the four reviewed ASMG perspectives fully address the accessibility limitations fundamental in traditional simulation environments, as they all maintain some degree of technical complexity or domain-specific knowledge requirements that continue to constrain widespread adoption. The recent emergence of Large Language Models presents a notable opportunity to realize the full potential of interface-facilitated approaches through natural language interaction capabilities.

The natural language opportunity: integrating LLMs with ASMG

The analysis of ASMG perspectives in “*Evolution of automatic simulation model generation*” section revealed that interface-facilitated approaches represent the most promising pathway for addressing simulation accessibility challenges. However, a critical limitation in current ASMG approaches is the poor adoption of effective natural language interfaces (NLIs) that would allow simulation developers to express modeling needs in familiar language. Son et al. (2003) and Son and Wysk (2001) introduced this concept 2 decades ago, envisioning systems where users interact with simulation software using everyday language, yet this vision remains largely unrealized.

Recent systematic literature reviews explicitly identify this lack of intuitive interfaces as a significant constraint Behrendt et al. (2025), particularly for simulation systems that aim to adapt to changing manufacturing environments. A systematic review by Cimino et al. (2025) further reinforces this perspective, suggesting that NLIs could substantially improve accessibility by allowing users to express simulation requirements without specialized terminology.

To understand the extent of this limitation, we conducted a comprehensive analysis of Interface-Facilitated ASMG approaches based on Son et al.'s (2001) original conceptualization of three distinct interface dimensions. As shown in Table 1, our analysis reveals a clear pattern: despite the theoretical framework being established over two decades ago, no implementation has successfully realized Natural Language Interface or Interactive Dialogue Interface capabilities in ASMG systems.

The comparative analysis demonstrates that Interface-Facilitated ASMG has been exclusively limited to graphic interface implementations for the past two decades. While these GUI-based approaches have achieved success in specific domains (Cimino et al., 2025), they maintain significant barriers, including requirements for simulation domain knowledge, structured data input formats, and limited accessibility for non-technical users.

The emergence of large language models presents a promising pathway to address this long-standing challenge. These models demonstrate unprecedented capabilities across diverse neural architectures (Chang et al., 2024; Shi et al., 2024), including understanding complex instructions (Patil & Gudivada, 2024), generating functional code (Jiang et al., 2024), adapting to specialized domains (Hadi et al., 2023), and providing detailed explanations (Chang et al., 2024). These capabilities align directly with the requirements for effective natural language interfaces in simulation contexts.

Table 1 Comparative analysis of interface-facilitated ASMG methodologies

Reference	Graphic	Natural language	Interactive dialogue
Almeida and Tilbury (2004)	Yes	No	No
Mueller et al. (2007)	Yes	No	No
Nasereddin et al. (2007)	Yes	No	No
Burnett et al. (2008)	Yes	No	No
Wy et al. (2011)	Yes	No	No
Wang et al. (2011)	Yes	No	No
Du et al. (2013)	Yes	No	No
Billiet and Stark (2022)	Yes	No	No
Steinbacher et al. (2023b)	Yes	No	No
Our approach	Yes	Yes	Yes

Despite this potential alignment, the integration of LLMs with simulation systems for automatic model generation remains in its infancy. Recent work by Zhang et al. (2025) demonstrates initial steps toward LLM use for simulation model generation in Model-Based Systems Engineering via X Language (Xie et al., 2024), though focused primarily on the product domain rather than manufacturing processes. Mata et al. (2025) present a comprehensive framework integrating ChatGPT with digital twin design for manufacturing systems. However, their approach centers on digital twin design methodology rather than the automatic generation of simulation models themselves, highlighting a distinct but complementary research direction. Adjacent domains have begun leveraging LLMs for complex technical tasks, including robotics navigation (Wang et al., 2024), industrial knowledge modeling (Lee & Su, 2023), and 3D printing optimization (Badini et al., 2023), as well as emerging applications in fault analysis systems (Wang et al., 2025) and autonomous robotics (Fan et al., 2025), suggesting transferable approaches for simulation applications.

Our methodology is a step forward in the integration of LLM technology with ASMG to unlock both natural language Interface and Interactive Dialogue Interface capabilities simultaneously. Unlike existing approaches that are limited to single-interface implementations, our framework demonstrates that LLMs serve as the technological opportunity needed to realize the complete three-dimensional interface-facilitated ASMG framework. This integration enables users to create simulation models using conversational requests without technical knowledge while supporting interactive clarifying conversations, handling incomplete specifications, and guiding users through model development processes.

This LLM-enhanced ASMG approach could help transform simulation accessibility while maintaining necessary analytical power. By potentially democratizing access to simulation technology across industrial domains, this approach may reduce development time while eliminating some specialized programming requirements. Such integration aligns with the broader research agenda for next-generation simulation systems that emphasize effective human-machine interaction in industrial contexts (Elbashaer et al., 2024), offering an opportunity may help bridge the persistent gap between simulation complexity and user accessibility.

Methodology

This section presents our novel methodology for integrating LLMs with ASMG, directly addressing RQ1 by demonstrating how natural language interfaces can enable non-expert

users to create simulation models for production planning. The methodology eliminates traditional barriers of specialized programming knowledge and formal modeling specifications through a four-phase framework that transforms conversational requests into executable simulation models.

Building on the identified gap in natural language interfaces for simulation, this section presents our methodology for integrating LLMs with ASMG. Our approach directly addresses the limitations highlighted by Behrendt et al. (2025) and Cimino et al. (2025) by creating a bridge between natural language understanding and simulation model construction, enabling the streamlining of industrial simulation models development through everyday language. The core methodological contribution relies on using an object-oriented simulation framework that provides flexibility and adaptability for production planning applications. The framework employs a sophisticated three-class structure (Input, Workstation, and System) with data-driven design principles that enable automatic model reconfiguration, dynamic resource instantiation, and seamless integration of diverse manufacturing scenarios without manual model reconstruction. This simulation architecture ensures reliability through validated parameter constraints and established discrete-event simulation principles while maintaining the computational accuracy required for industrial decision support.

As shown in Fig. 1, the framework consists of an end-to-end process flow divided into two distinct phases: the LLM Phase and the Simulation Phase, with each phase further subdivided into two steps. The LLM Phase addresses the documented accessibility gap in simulation technology by transforming natural language expressions into structured specifications that feed our advanced simulation model. The Simulation Phase leverages our object-oriented architecture where the Workstation class encapsulates manufacturing resources with built-in processing capabilities, the System class orchestrates material flow and real-time state management, and the Input class manages seamless data integration. This architectural design enables the framework's flexibility, supporting adaptive process mapping, scalable system configurations, and dynamic parameter adjustment, while LLMs provide the natural language interface that can democratize access to these sophisticated modeling capabilities.

LLM phase

The LLM Phase handles the transformation from natural language input to structured data representation, consisting of two critical steps: User Request (frontend) and Knowledge Extraction (backend).

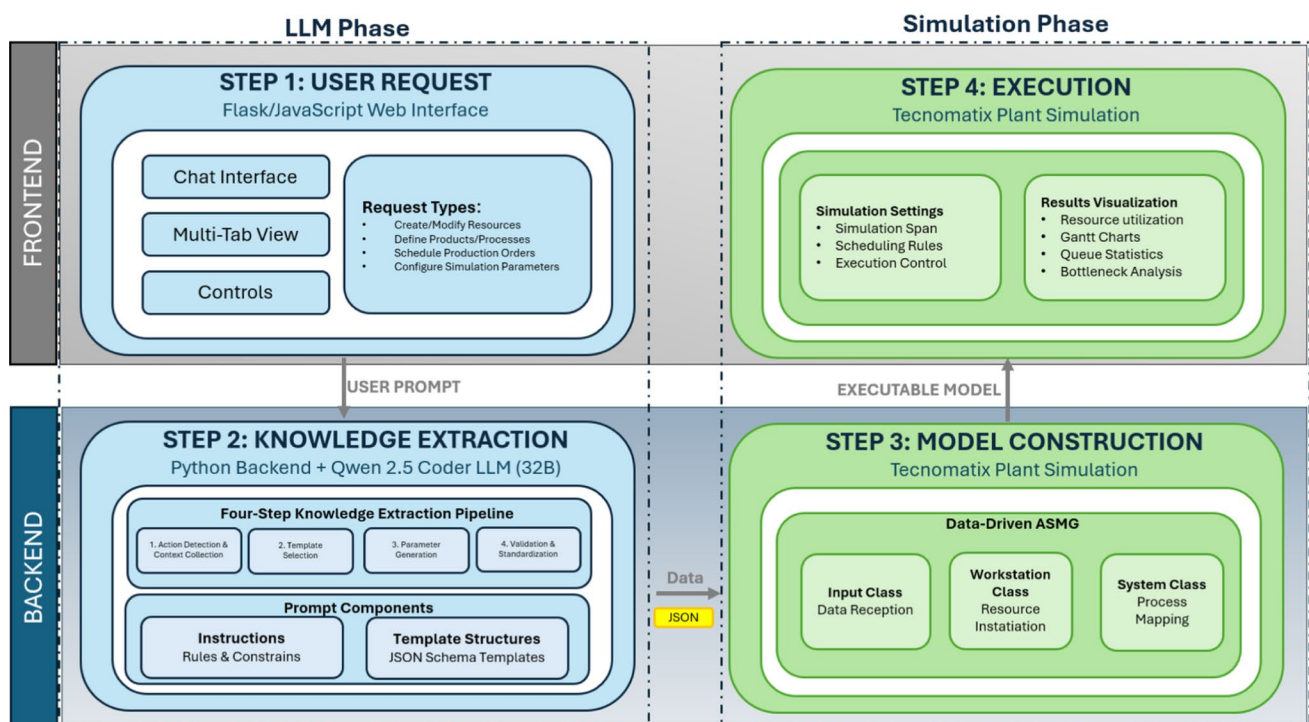


Fig. 1 Natural language-driven simulation framework showing the four-phase process flow from user request to simulation execution

Step 1: user request

The first phase begins with users expressing their simulation needs in natural language through an intuitive web-based interface developed using Flask and JavaScript. This approach eliminates the requirement for specialized simulation knowledge or programming skills. As shown in Step 1 of the framework diagram, the frontend includes a chat interface, multi-tab view, and controls for interacting with the system. The interface handles various request types including create/modify resources, define products/processes, schedule production orders, and configure simulation parameters.

The system implements a chat-like interface resembling interaction with a virtual assistant, processing commands asynchronously via Asynchronous JavaScript and XML (AJAX) requests. User requests are captured as unstructured text and transmitted to the knowledge extraction component, forming the starting point of the model generation pipeline.

Step 2: knowledge extraction

The knowledge extraction phase, powered by Qwen 2.5 Coder (a 32B parameter LLM) within a Python backend, transforms linguistic content into structured domain-specific representations that directly satisfy the data requirements of the ASMG (Step 3). This critical phase bridges the gap between user intent expressed in natural language and the precise specifications required by simulation systems.

A key innovation in this approach is implementing a Four-Step Knowledge Extraction Pipeline that applies established prompt engineering principles to mitigate common LLM limitations such as hallucinations, inconsistencies, and ambiguous outputs (Ji et al., 2023; Tonmoy et al., 2024). The pipeline follows structured prompting methodologies (Cheng et al., 2023) and incorporates template-based approaches that constrain LLM generation through predefined schemas, ensuring consistent and reliable parameter extraction for simulation model development.

The complete implementation of this pipeline is detailed through both algorithmic pseudocode (Algorithm 1, “Appendix 1”) and architectural visualization (Fig. 2), providing comprehensive documentation of the logical flow and structural components of the extraction process. The framework implements a hybrid approach that strategically combines deterministic rule-based processing with AI-driven inference to achieve both reliability and flexibility: deterministic operations handle systematic tasks requiring consistency and speed, while LLM inference addresses the complex semantic understanding required for natural

language processing. This structure enables the use of pre-trained LLMs without domain-specific fine-tuning, as the system provides industrial domain knowledge through structured prompting with two major Prompt Components: Instructions (rules and constraints) and template structures (JSON schema templates). The open-source model selection supports reproducibility while leveraging existing code generation capabilities rather than requiring specialized training for simulation tasks. The approach implements core principles from retrieval-augmented generation (RAG) methodologies (Lewis et al., 2020) by systematically retrieving and selecting appropriate template structures based on the detected action type, ensuring that LLM outputs conform to predefined JSON schemas required for simulation model construction.

The pipeline’s operational logic follows prompt engineering best practices (White et al., 2023) through systematic progression of four distinct phases, each designed to address specific aspects of structured knowledge extraction from natural language inputs:

1. *Action detection and context collection (rule-based)* This deterministic phase employs keyword recognition algorithms and pattern matching to analyze natural language requests, mapping linguistic patterns to formal action categories through predefined dictionaries and taxonomies. The rule-based approach ensures consistent inference times and provides deterministic mapping without LLM overhead. The pipeline begins by analyzing the natural language request to identify the intended operation type, mapping linguistic patterns to formal action categories through keyword recognition. Simultaneously, it collects contextual information about the current manufacturing system state through the Data-Access module interface, creating the foundation for accurate parameter identification.
2. *Template selection and prompt construction (rule-based)* Operating as a deterministic process, this phase performs template library lookup from 18 predefined JSON schemas based on the classified action type. This implements RAG principles by retrieving the most relevant structural template for the identified action, similar to knowledge retrieval approaches (Gao et al., 2023) that select appropriate external information to guide generation. The deterministic combination of request, context, and template structures ensures reproducible prompt generation with consistent formatting. Based on the detected action type, the system selects the appropriate structured JSON schema template defining the expected output format. These templates are specifically designed to match the data schema required by the

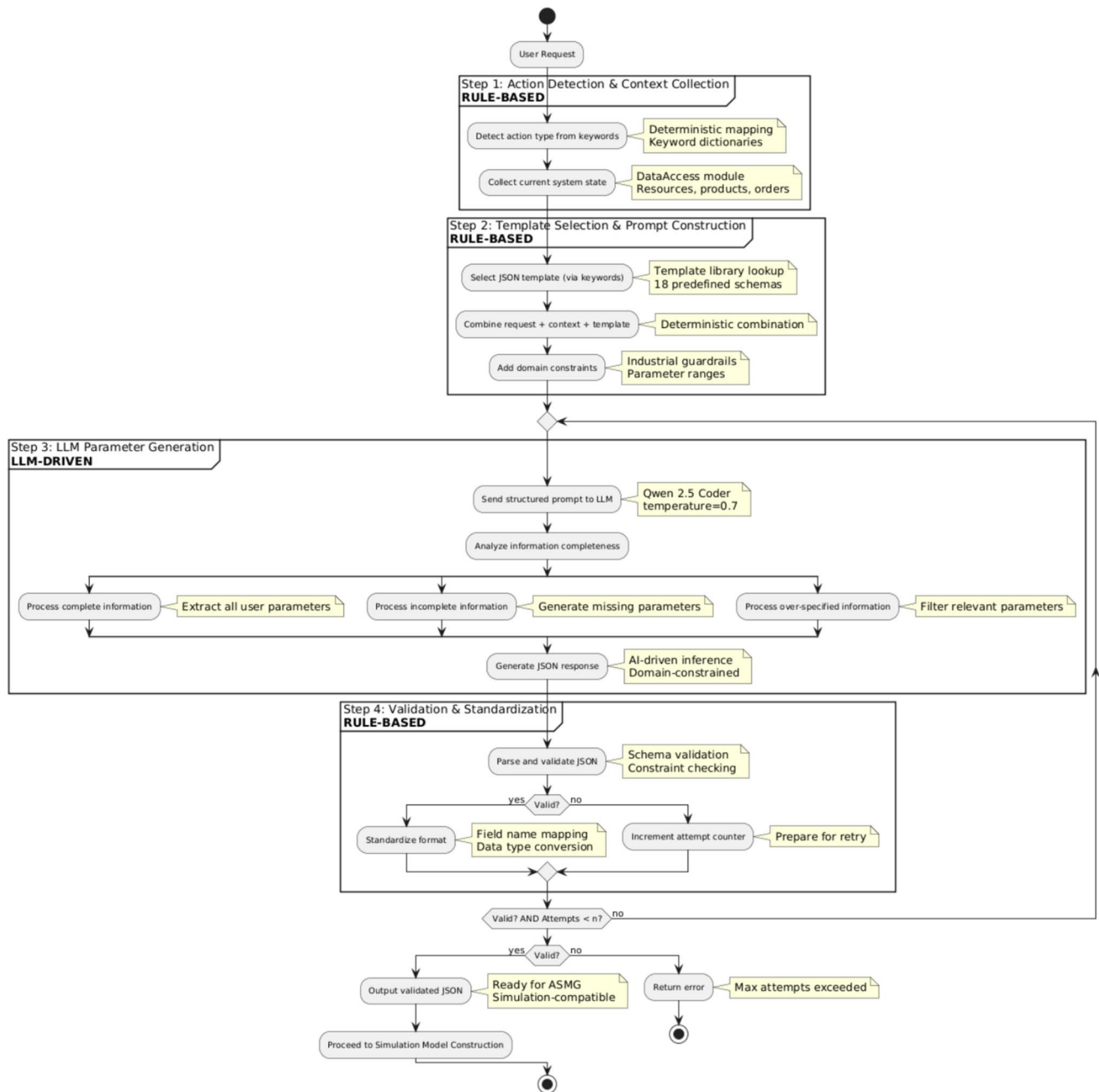


Fig. 2 Four-step knowledge extraction pipeline architecture showing the integration of rule-based and LLM-driven components

ASMG components in Step 3, ensuring perfect alignment between knowledge extraction and simulation model construction.

The instructions component provides comprehensive industrial domain rules and constraints that establish fundamental knowledge about manufacturing systems, including typical resource parameters, processing time ranges, and production planning constraints. These instructions serve as knowledge guardrails following prompt engineering principles (Si et al., 2022) that

guide LLM behavior through explicit constraints and domain-specific context, preventing the generation of unrealistic or inappropriate parameter values by embedding manufacturing expertise directly into the prompt structure.

The system then constructs a comprehensive prompt combining the user request, system context, template structures, and instructions to guide the LLM. This deterministic prompt construction process includes industrial guardrails specifying parameter ranges that

constrain LLM outputs to realistic industrial values through structured prompt design.

3. *LLM-based parameter generation (LLM-driven inference)* This phase represents the core LLM inference process, where Qwen 2.5 Coder (configured with temperature = 0.7 for balanced creativity and consistency) processes structured prompts to generate domain-constrained JSON responses. The approach follows established prompt engineering methodologies by providing clear instructions, structured templates, and domain constraints that guide the model toward generating appropriate outputs. The Qwen 2.5 Coder LLM processes the prompt to generate a structured JSON response containing all necessary parameters. The system's behavior adapts to the level of detail in the user's request:

- *Complete information scenario* when users provide all required information, the LLM strictly extracts and preserves these explicit parameters without modification.
- *Incomplete information scenario* only when parameters are omitted does the LLM generate plausible supplementary values based on the provided industrial domain knowledge, clearly distinguishing between user-provided and system-generated parameters.
- *Over-information scenario* when users provide excessive or non-relevant details, the system selectively extracts only the information needed for simulation model development, filtering out extraneous data that doesn't align with the ASMG requirements.

Failure handling mechanisms include retry logic (maximum 3 attempts), fallback to simplified templates for complex requests, and escalation to human review queues when validation thresholds are not met.

4. *Validation and Standardization (rule-based with retry logic)* This deterministic phase implements schema validation and constraint checking through programmatic verification of JSON structure, data types, and value ranges. Processing typically completes with automated retry mechanisms triggered when validation fails. Importantly, this validation focuses exclusively on structural correctness and format compliance rather than content verification—checking that required fields are present, data types are correct, and JSON syntax is valid, without evaluating the semantic accuracy or industrial feasibility of the generated parameters. The generated JSON undergoes structural validation and format standardization to ensure compatibility with the

simulation environment through HTTP RESTful communication. This step focuses on verifying the syntax and structure of the output rather than semantics, ensuring consistent field names, appropriate data types, and adherence to the expected schema required by the simulation framework. When generated JSON fails validation, the system automatically reinvokes the LLM with refined prompts that address the specific validation errors, ensuring robust parameter generation despite initial failures through iterative correction mechanisms.

The pipeline architecture integrates rule-based processing with LLM inference through a four-step process documented in Algorithm 1 (“Appendix 1”). Deterministic components handle action classification and template selection, while LLM inference generates structured parameters based on user inputs and industrial constraints.

The framework employs structured prompt engineering combining industrial domain instructions with predefined JSON schema templates. Domain instructions provide manufacturing context including typical parameter ranges and operational constraints. Template structures define the expected output format for 18 supported action types, ensuring compatibility with the simulation model construction phase.

The system handles three input scenarios: complete information (parameters preserved as provided), incomplete information (missing values generated using domain knowledge), and over-information (relevant parameters extracted, extraneous data filtered). Generated outputs undergo structural validation to verify JSON syntax, data types, and schema compliance before transmission to the simulation phase.

Examples of extracted JSON specifications for resources, products, and orders are provided in “Appendix 2”, with their practical application and effectiveness demonstrated through the comprehensive use case presented in “Results” section.

Simulation phase

The Simulation Phase transforms the structured specifications into functional simulation models and executes them to provide insights, consisting of two technical steps: model construction (backend) and simulation execution (frontend).

Step 3: simulation model construction

The model construction phase implements a data-driven ASMG approach using Tecnomatix Plant Simulation, a specialized discrete-event simulation platform particularly

well-suited for automated model generation. Plant Simulation's object-oriented architecture, robust API capabilities, and built-in manufacturing components make it a preferable environment for ASMG implementation, as it supports programmatic model creation through its SimTalk scripting language and HTTP REST interfaces.

The proposed three-class architecture introduces several distinguishing characteristics compared to conventional modular simulation frameworks used in manufacturing contexts. The architectural design incorporates notable elements: (1) automated correspondence between LLM-produced parameters and simulation components through the Input class's verification and creation procedures, (2) flexible resource setup within the Workstation class that accommodates varying levels of specification detail from user inputs, and (3) the System class's capability to coordinate material flows and resource interactions by dynamically constructing routing logic from process specifications rather than relying on predefined configurations. These characteristics support the framework's integration with natural language processing, allowing diverse user expressions to produce functional simulation models regardless of their technical precision.

The simulation models focus on capturing key production dynamics including resource utilization, throughput capacity, bottleneck identification, and queue behavior with medium-to-high fidelity. This targeted accuracy level determines the specific data parameters required from Step 2, balancing model complexity with practical utility. Higher accuracy in cycle time prediction and resource state modeling requires more detailed JSON specifications, while approximate capacity planning can function with simpler parameter sets, as defined in Cimino et al. (2025) for developing adaptive simulation models for production planning purposes.

To ensure reproducible and reliable simulation results, the framework implements standardized configuration parameters. Processing times utilize uniform distributions with $\pm 10\%$, reflecting realistic manufacturing variability. Simulation duration is configurable through natural language specifications or intelligent defaults based on manufacturing scenario complexity. Warm-up periods are based on system characteristics, typically requiring 4–6 h of simulated time for production systems starting from empty states to reach steady-state operations. The framework utilizes controllable

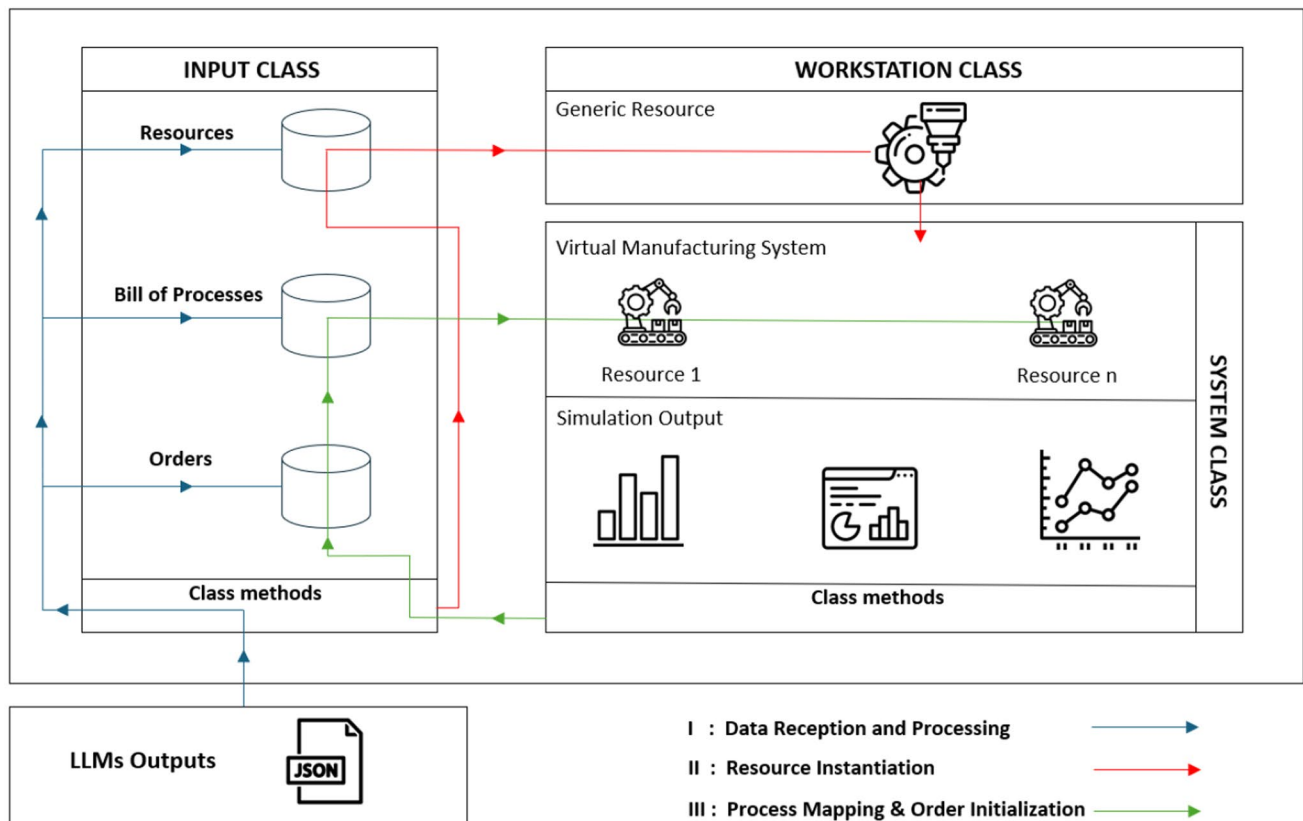


Fig. 3 ASMG object-oriented framework showing the three-class architecture and primary data flows for simulation model generation in plant simulation

random seeds for reproducible comparative analyses while maintaining stochastic realism in manufacturing processes.

Input to this phase consists of structured JSON specifications received through HTTP RESTful communication, leveraging the simulation platform's ability to directly consume external data sources without manual intervention. As shown in Fig. 3, the implementation architecture is built around three primary classes that work together in the simulation environment, with each component reflecting established ASMG methodological perspectives:

1. The Input class serves as the gateway between Knowledge Extraction and the simulation environment, receiving JSON data, validating it against expected templates, and creating workstation objects corresponding to each resource using the platform's library of manufacturing components. This component aligns with data-centric ASMG principles through its emphasis on structured data processing, validation protocols, and systematic transformation of external information into simulation-compatible formats.
2. The Workstation class encapsulates individual manufacturing resources, utilizing built-in processing, transport, and storage objects to simulate processes with appropriate timing, manage scheduling rules, and track resource states. This component exemplifies knowledge-based ASMG characteristics by incorporating reusable domain expertise within modular structures, where each workstation maintains embedded understanding of manufacturing operations, resource limitations, and typical processing behaviors found in production environments.
3. The System class orchestrates the entire simulation by managing workstation interactions, leveraging visualization capabilities, integrated statistics collection, and material flow control features. This component combines elements from multiple ASMG perspectives: it employs algorithmic techniques for dynamic scheduling and routing decisions while providing programmatic interfaces that connect with the natural language processing capabilities, thereby bridging computational optimization with user accessibility.

The integration of multiple methodological perspectives within a unified framework represents a departure from typical ASMG implementations that concentrate on individual approaches. Data-centric systems, while efficient at processing structured information, generally lack the adaptability required for natural language interpretation. Similarly, interface-focused implementations may provide user accessibility but often lack the computational sophistication necessary for realistic manufacturing simulation. Our

architectural approach attempts to harmonize these different strengths, creating a framework that balances technical capability with practical accessibility.

The model generation process follows these systematic steps (Fig. 3):

1. *Data reception and processing* the input class receives JSON data containing specifications for Resources, Bill of Processes, and Orders.
2. *Resource instantiation* for each resource, the Input class creates a corresponding Workstation object with matching properties.
3. *Process mapping and order initialization* the system class maps operations to appropriate Workstation objects and creates production orders.

The data-driven ASMG model construction approach, in Step 3, aligns with the ASMG developing perspectives by implementing an object-oriented architecture that automatically generates simulation models from structured specifications (Cimino et al., 2025). The three-class framework ensures modularity and scalability while maintaining the accuracy and responsiveness advantages demonstrated in ASMG literature for manufacturing applications.

Step 4: simulation execution

The execution component implements a bidirectional API layer between the natural language processing system and the simulation platform, providing standardized endpoints for model control and results aggregation tailored to production planning needs. The execution engine initializes the model with parameters like simulation duration and random seed values, performs the simulation, and processes results into manufacturing KPIs.

Rather than creating separate visualizations, the system directly opens the simulation platform's native analysis tools and dashboards (gantt charts, utilization statistics, and bottleneck analyzers) in response to natural language requests, leveraging the platform's comprehensive visualization capabilities.

Results

This section evaluates our natural language-driven production planning framework through two complementary approaches that systematically address both research questions posed in "Introduction" section. The practical application (in "Practical application" section) demonstrates how the framework enables non-expert users to generate functional simulation models through conversational interaction,

directly addressing RQ1 by illustrating the complete transformation from natural language input to actionable production planning insights. The performance testing and baseline comparison (in “[Performance testing](#)” section) addresses RQ2 by quantifying response times and processing efficiency across different operation types while providing empirical evidence of accessibility improvements through controlled comparison with traditional ASMG approaches.

Practical application

The practical application of our Natural Language-Driven Simulation framework in a production planning context demonstrates the four-step methodology described in “[Methodology](#)” section: User Request, Knowledge Extraction, Simulation Model Construction, and Simulation Execution. Through this developing use case, we illustrate how the framework processes natural language inputs to

generate functional simulation models for manufacturing decision support.

Step 1: user request

In the User Request step, domain experts interact with the system through a web-based interface with multiple tabs for comprehensive system management. The interface includes Order Management for handling production orders, Products Management for defining bill of processes, Resources Management for configuring machines and operators, Simulation Control for setting parameters, and the AI Interaction Tab where users can perform all operations using natural language rather than form-based inputs.

Through the AI Interaction Tab, users can express a wide range of operations in natural language, including:

- *Resource operations* add resource, modify resource, delete resource

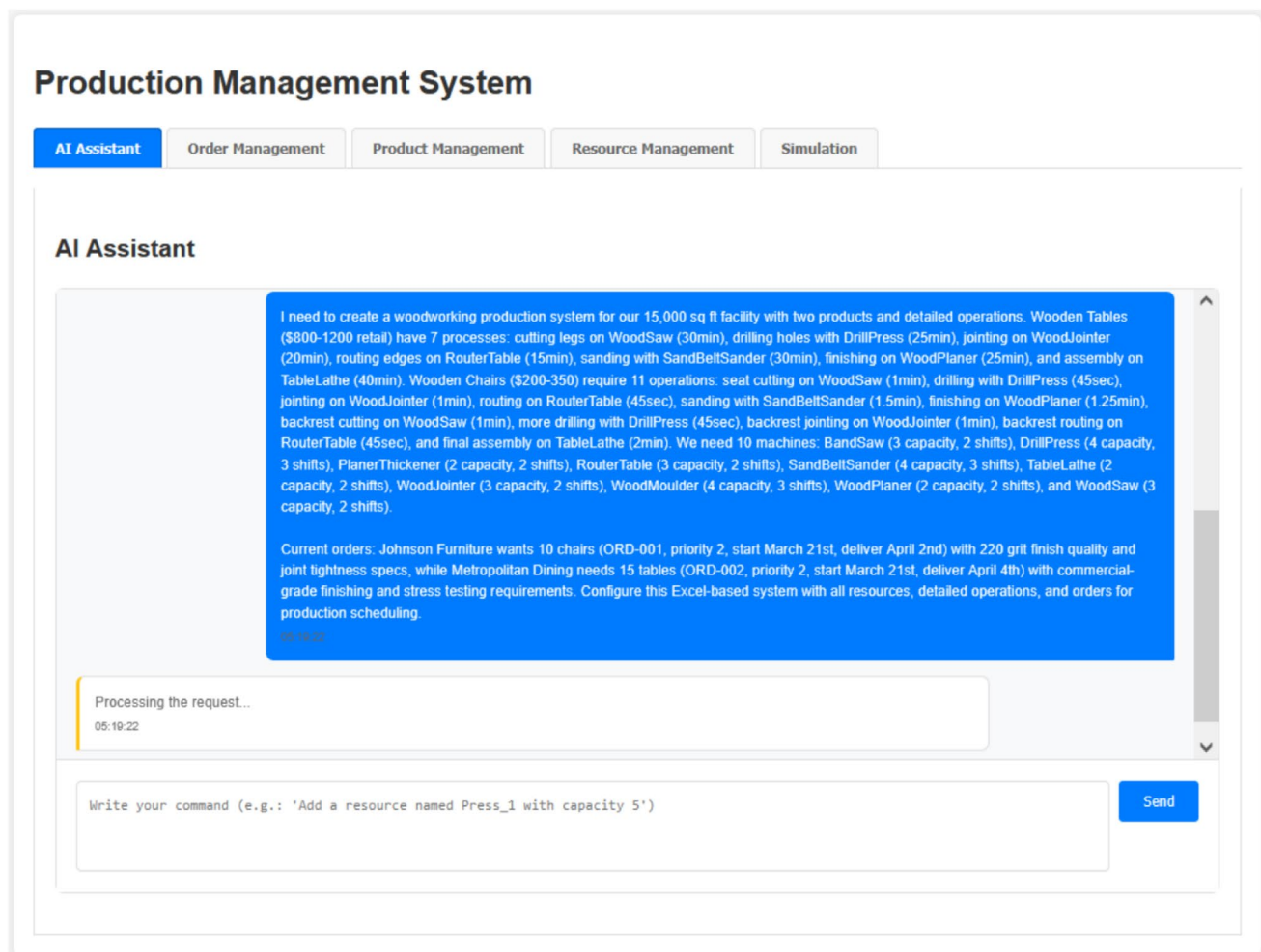


Fig. 4 The natural language interface showing a user request to create a woodworking production system with over specifications

- *Product operations* add product, modify product, delete product
- *Order operations* add order, modify order, delete order
- *Simulation operations* start simulation, stop simulation
- *Analysis operations* show utilization level, show queue level, show queue time, show gantt
- *Configuration operations* set scheduling rules (e.g., FIFO, SPT, EDD)

Beyond these individual operations, the system also supports compound requests that combine multiple operations in a single natural language instruction. For example, users can request to “create a manufacturing system” which initiates a sequence of coordinated actions including resource creation, product definition, and order scheduling. This capability allows users to express complex intentions in a single conversational request rather than executing multiple individual commands.

For this use case, the user provides a request with over and disorganized information as shown in Fig. 4:

“I need to create a woodworking production system for our 15,000 sq ft facility with two products and detailed operations. Wooden Tables (\$800–1200 retail) have 7 processes: cutting legs on WoodSaw (30 min), drilling holes with DrillPress (25 min), jointing on WoodJointer (20 min), routing edges on RouterTable (15 min), sanding with SandBeltSander (30 min), finishing on WoodPlaner (25 min), and assembly on TableLathe (40 min). Wooden Chairs (\$20–35) require 11 operations: seat cutting on WoodSaw (1 min), drilling with DrillPress (45 s), jointing on WoodJointer (1 min), routing on RouterTable (45 s), sanding with SandBeltSander (1.5 min), finishing on WoodPlaner (1.25 min), backrest cutting on WoodSaw (1 min), more drilling with DrillPress (45 s), backrest jointing on WoodJointer (1 min), backrest routing on RouterTable (45 s), and final assembly on TableLathe (2 min). We need 10 machines: BandSaw (3 capacity, 2 shifts), DrillPress (4 capacity, 3 shifts), PlanerThickener (2 capacity, 2 shifts), RouterTable (3 capacity, 2 shifts), SandBeltSander (4 capacity, 3 shifts), TableLathe (2 capacity, 2 shifts), WoodJointer (3 capacity, 2 shifts), WoodMoulder (4 capacity, 3 shifts), WoodPlaner (2 capacity, 2 shifts), and WoodSaw (3 capacity, 2 shifts).

Current orders: Johnson Furniture wants 10 chairs (ORD-001, priority 2, start March 21st, deliver April 2nd) with 220 grit finish quality and joint tightness specs, while Metropolitan Dining needs 15 tables (ORD-002, priority 2, start March 21st, deliver April 4th) with commercial-grade finishing and stress testing requirements.”

This request exemplifies an over-information scenario, containing extensive specifications including detailed processing times, resource capacities, shift patterns, pricing

information, facility details, quality requirements, and operational sequences. The system must selectively extract only simulation-relevant parameters while filtering out extraneous business context and non-essential details for model generation.

Step 2: knowledge extraction

In the Knowledge Extraction step, the system processes the natural language request to extract the structured information required for model creation. Following the methodology described in “[Step 2: knowledge extraction](#)” section, the system combines the user request with context information and template structures before sending it to the LLM for processing.

The woodworking request from Step 1 exemplifies an over-information scenario containing extensive specifications mixed with business details irrelevant to simulation modeling. The system applies the Four-Step Knowledge Extraction Pipeline (Algorithm 1, “[Appendix 1](#)”) to parse through facility descriptions (“15,000 sq ft facility”), pricing information (“800–1200 retail”), quality specifications (“220 grit finish quality”), and operational details to selectively extract simulation-relevant parameters while discarding extraneous information.

The LLM transforms the unstructured request into a series of structured JSON specifications that define the complete manufacturing system. This transformation follows the algorithmic framework where the system detects the compound action “create manufacturing system,” selects appropriate templates for resources, products, and orders, constructs domain-constrained prompts, and generates validated JSON outputs that preserve essential manufacturing parameters while filtering non-simulation elements. The LLM’s template-guided architecture ensures that all required parameters are generated in a format compatible with the simulation environment.

In this case, the LLM generates three distinct types of structured specifications that directly correspond to the three-class ASMG architecture described in “[Step 3: simulation model construction](#)” section (Fig. 3):

1. Resource specifications defining 10 woodworking machines with domain-appropriate properties including machine types (band saws, drill presses, sanders, etc.), energy consumption values, processing capacities, and shift patterns typical for woodworking facilities, which are processed by the Input class for workstation instantiation. The system transforms user descriptions like “WoodSaw (3 capacity, 2 shifts)” into complete JSON structures with resourceName: “WoodSaw”, capacity:

- 3, shiftsNumber: 2, plus system-generated realistic values for workingConsumption and waitingConsumption based on woodworking machinery standards, as referenced in Listing “Appendix 2.1”.
2. Product specifications defining the WoodenTable and WoodenChair products with their respective process sequences, including realistic operation names, appropriate pre-processing, processing, and post-processing times for each woodworking operation, which enable the System class to perform process mapping and material flow orchestration. Complex user descriptions such as “cutting legs on WoodSaw (30 min)” are systematically converted into structured operations with Operation: “CuttingLegs”, Processing Time: 30.0, Workstation: “WoodSaw”, along with generated setup and cleanup times based on woodworking domain knowledge, as referenced in Listing “Appendix 2.2”.
3. Order specifications for the requested batch production of 10 chairs and 15 tables, including reasonable production start dates and expected delivery dates, which are used by the System class for order initialization and production scheduling. Business-oriented requests like “Johnson Furniture wants 10 chairs” are filtered and standardized into Customer-Order Nr.: “ORD-001”, Product Type: “WoodenChair”, Quantity: 10, with non-essential details like quality requirements removed during the extraction process, as referenced in Listing “Appendix 2.3”.

The parameter generation process demonstrates how the system handles incomplete specifications through industrial domain constraints embedded in the prompt templates. When users omit specific values (such as energy consumption for machinery), the system applies manufacturing knowledge to generate realistic parameters within appropriate ranges, for instance, woodworking equipment typically operates between 3–8 kW, enabling the system to produce plausible baseline values that ensure model functionality.

The interaction between these three specification types and their processing by the ASMG framework is illustrated in the systematic steps detailed in “[Step 3: simulation model construction](#)” section, where the Input class receives and validates these JSON specifications, the Workstation class encapsulates the resource definitions, and the System class orchestrates the process mapping and order management. The system converts these linguistic specifications into simulation parameters through a structured template approach that ensures consistent interpretation regardless of phrasing. This standardization process, implemented through the StandardizeParameters function in Algorithm 1, maintains parameter consistency across different linguistic expressions while preserving essential manufacturing information required for accurate simulation modeling. The complete JSON specifications in “[Appendix 2](#)” demonstrate how diverse natural language descriptions converge into standardized formats suitable for automatic simulation model construction.

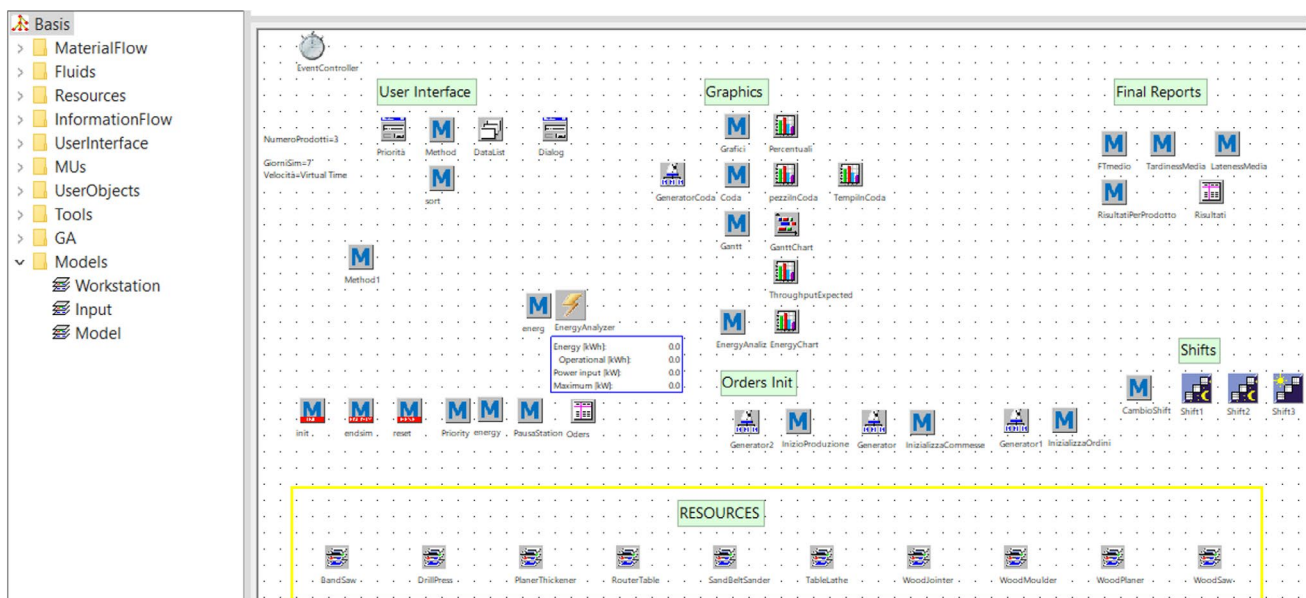


Fig. 5 The automatically generated simulation model showing workstation objects (highlighted in yellow) created from JSON specifications, with supporting objects for material flow control, visualization, and KPI generation

Step 3: simulation model construction

In the Simulation Model Construction step, the system transforms the structured specifications into a functional simulation model focused on production planning. Following the methodology described in “[Step 3: simulation model construction](#)” section, the system constructs a discrete-event simulation model in Tecnomatix Plant Simulation using an object-oriented approach. The simulation model is specifically designed for production planning purposes, enabling analysis of resource utilization, throughput capacity, bottleneck identification, and queue behavior. These capabilities provide manufacturing stakeholders with crucial insights for optimizing production schedules, resource allocation, and workflow sequencing. For the use case, as shown in Fig. 5, the ASMG component programmatically instantiates simulation objects based on the JSON specifications received from the Knowledge Extraction step.

The automatically generated simulation model demonstrates the System class implementation described in “[Step 3: simulation model construction](#)” section. The simulation environment contains the workstation objects (highlighted in yellow as RESOURCES) and a comprehensive set of supporting objects that ensure complete simulation model functionality. The System class orchestrates multiple components including material flow control objects, visualization elements for real-time monitoring, data collection structures for KPI calculation, and integrated reporting mechanisms. The object tree on the left shows the complete

model hierarchy, where each workstation object encapsulates the resource specifications generated from the JSON inputs, while additional system objects provide automatic graph generation capabilities, flexible process mapping functionality, and performance metric calculation methods. This architecture enables the framework to automatically instantiate a complete, executable simulation environment that supports comprehensive production planning analysis without manual configuration.

The system automatically configures appropriate connections between resources based on the process sequences defined for each product, creating a complete material flow network. Each resource object simulates specific production operations including processing times, setup requirements, and material transformations based on the specifications generated by the LLM. Beyond the visible workstation objects, the system establishes a comprehensive production environment with:

- Material flow paths connecting workstations according to operational sequences
- Queue structures with appropriate capacity limitations
- Resource state tracking for availability analysis
- Product routing logic based on bill of process specifications
- Data collection objects for performance metric analysis
- Visualization components for real-time status monitoring

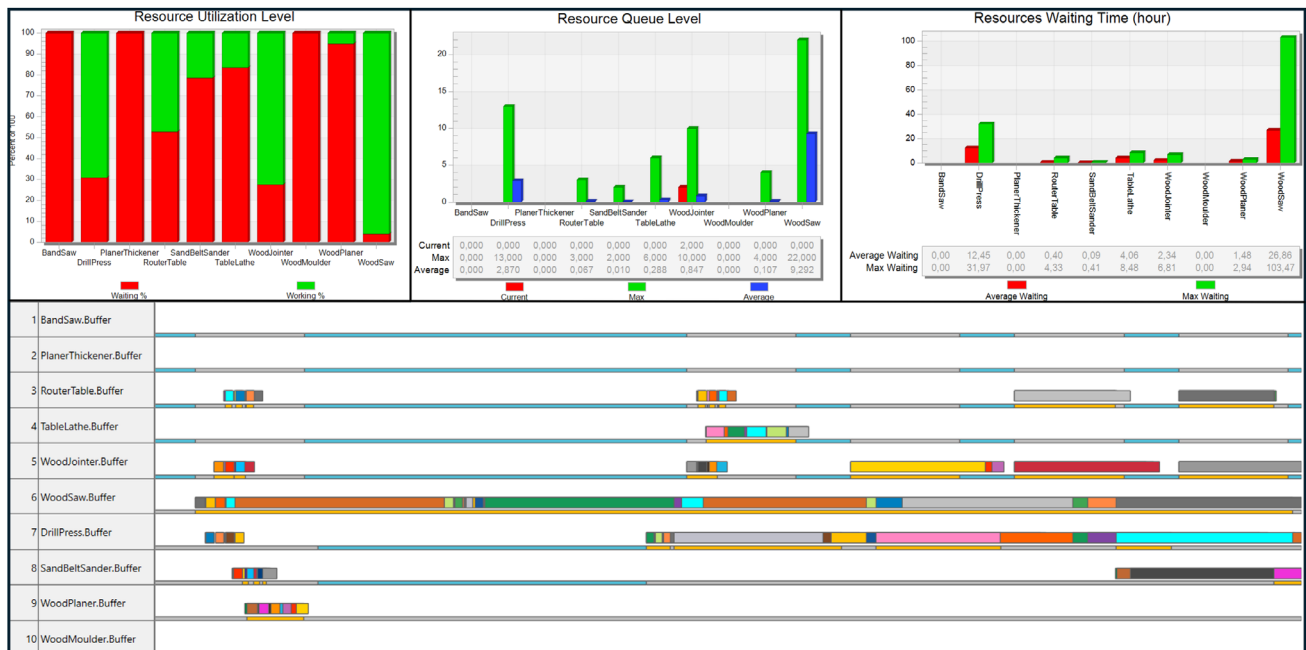


Fig. 6 Simulation execution results showing resource utilization, queue statistics, and production schedules generated from the automatically constructed model

Together, these components create an integrated production planning simulation that enables comprehensive system analysis from a model generated entirely through natural language interaction.

Step 4: simulation execution and analysis

Once the simulation model is constructed, the system executes it and generates comprehensive results for analysis, as shown in Fig. 6.

The simulation execution provides multiple visualizations and metrics to support decision-making in the production planning context:

1. *Resource utilization analysis* the system identifies bottleneck stations through utilization statistics, revealing that the WoodSaw and SurfaceSander workstations have the highest utilization rates in the current configuration. This analysis enables targeted capacity improvement interventions that would maximize overall system throughput with minimal investment.
2. *Queue statistics* the visualization shows work-in-progress accumulation points, particularly before high-utilization resources. These queue statistics help production planners determine appropriate buffer sizes and identify flow imbalances that could be addressed through process re-sequencing or parallel operations.
3. *Waiting time analysis* the system calculates the ratio of waiting time to processing time for each product, revealing process flow inefficiencies. For the wood-working system, the analysis shows that products spend significant time waiting before certain operations, indicating opportunities for resource reallocation or scheduling optimization.
4. *Gantt chart visualization* the time-based visualization of resource allocation and order progression provides production schedulers with a clear view of expected completion timelines and resource conflicts. This visualization helps identify opportunities for improved scheduling algorithms or batch size adjustments.

The simulation results provide actionable insights for manufacturing decision support, including:

- Identification of capacity constraints limiting overall system throughput
- Quantification of resource utilization imbalances across the production system
- Visualization of production order completion timelines for delivery planning

- Analysis of queue accumulation points requiring attention or redesign
- Evaluation of current scheduling rules against potential alternatives

This use case demonstrates how our natural language-driven simulation framework transforms conversational requests into valuable production planning insights. The system generates functional simulation models that provide comprehensive decision support capabilities while requiring minimal technical input from users.

Performance testing

To establish baseline performance metrics for the natural language-driven simulation framework and address the need for comparative evaluation across manufacturing domains, we conducted systematic testing across all supported action types. This evaluation aimed to quantify system response times, assess processing consistency across different manufacturing sectors, and provide empirical comparison with traditional ASMG approaches through controlled experiments with non-expert users.

Testing methodology and metrics

Each action type represents a distinct operation within the framework's capability spectrum. Performance measurements encompass the complete processing pipeline from natural language input submission through structured JSON output generation and transmission to the simulation environment. The action types are categorized as follows:

- *Resource management actions* (Add/Modify/Delete Resource): Operations that create, update, or remove manufacturing resources within the simulation environment
- *Product management actions* (Add/Modify/Delete Product): Operations defining product specifications and their associated bill of processes
- *Order management actions* (Add/Modify/Delete Order): Operations handling production order creation and scheduling parameters
- *Simulation control actions* (Start/Stop Simulation, Set Scheduling Rules): Commands initiating simulation execution, termination, or configuring scheduling algorithms (FIFO, SPT, EDD)
- *Analysis actions* (Show Utilization/Queues/Queue Times/Gantt/Bottlenecks): Queries requesting visualization and performance metrics

- *Complex system creation* Compound operations generating complete manufacturing systems with multiple resources, products, and orders

Response times were measured from initial user request submission (Step 1) through complete knowledge extraction (Step 2) until successful data transmission to the simulation environment via Ethernet connection. These measurements exclude subsequent simulation model construction (approximately 5 s) and execution time (1–10 s depending on simulation span).

Experimental setup

The experimental infrastructure consisted of two computational environments connected via wired Ethernet (1 Gbps). The LLM inference server utilized an NVIDIA A100 80GB GPU, Intel i9-13900K processor, and 256GB RAM, running Ubuntu 20.04 LTS with PyTorch 2.4 and CUDA 12.4/cuDNN 9.1 for Qwen 2.5 Coder execution. The backend processing system operated on standard laptop hardware (AMD Ryzen 5 processor, 8GB RAM, 2GB GPU).

Testing protocols involved multiple repetitions per action type using standardized prompts provided by non-expert users. Statistical measures were calculated as follows:

- *Tests (n)* Number of independent test repetitions performed
- *Min time* Minimum observed response time across all tests (seconds)
- *Max time* Maximum observed response time across all tests (seconds)
- *Avg time (μ)* Arithmetic mean of all response times (seconds)
- *Std Dev (σ)* Standard deviation measuring response time variability (seconds)

Performance results

The performance results presented in Table 2 and Fig. 7 reveal distinct patterns correlating response times with operational complexity. Simple control operations (Start/Stop Simulation, Analysis Actions) demonstrate consistent performance with minimal variability, as these operations bypass complex LLM processing and directly interface with the simulation environment.

Data management operations show moderate to high variability depending on the complexity of knowledge extraction required. Resource and order management operations exhibit intermediate processing times ($\mu = 8\text{--}11$ s) with variability influenced by parameter complexity and specification detail. Product management operations display the highest variability among standard operations ($\mu = 20\text{--}24$ s, $\sigma = 3.8\text{--}5.2$ s), directly correlating with the number of manufacturing operations in the bill of processes and the complexity of operation parameters that must be extracted and validated.

Table 2 Performance testing results across all action types

Action Type	n	Min	Max	Avg	Std	Example prompt	Influencing factors
Add resource	15	8	15	11	2.8	"Add a CNC machine with double capacity"	Resource parameters complexity
Modify resource	12	7	12	9	1.5	"Change the capacity of DrillPress to 5"	Modification scope
Delete resource	10	6	10	8	1.1	"Remove the drilling machine"	—
Add product	14	18	32	24	5.2	"Create a wooden chair with 5 operations"	Number of operations, BOP complexity
Modify product	11	15	26	20	3.8	"Change assembly time to 3 min"	Process modification complexity
Delete product	8	8	12	10	1.2	"Delete the wooden table product"	—
Add order	16	8	14	11	1.8	"Create order ORD-003 for 20 units of turbine"	Order specification detail
Modify order	9	7	12	9	1.4	"Change ORD-001 quantity to 15"	Parameter modification
Delete order	7	6	10	8	1.0	"Cancel order ORD-002"	—
Start simulation	12	4	6	5	0.6	"Show me a simulation of the plant"	—
Stop simulation	8	3	5	4	0.5	"I want to stop the simulation now"	—
Set scheduling rules	10	4	7	5	0.8	"Set scheduling to FIFO"	—
Show utilization	15	3	5	4	0.6	"Show utilization for all machines"	—
Show queues	10	3	5	4	0.5	"Show me a graph of all queues in the production line"	—
Show queue times	8	3	5	4	0.6	"Show waiting times in queues"	—
Show gantt chart	12	3	6	4	0.8	"Display production schedule"	—
Show bottlenecks	9	3	5	4	0.5	"Show system bottlenecks"	—
Create manufacturing system	13	200	420	320	48.0	"Realize a manufacturing system that produces Pandoro and Panettone. Create 13 resources"	System complexity, number of resources, BOP

Complex system creation represents the most computationally intensive operation ($\mu = 320$ s, $\sigma = 48.0$ s), reflecting the compositional nature of generating complete manufacturing systems with multiple resources, detailed product specifications, and scheduling parameters. The substantial variance observed across test iterations demonstrates how system complexity factors, including total number of resources, bill of process intricacy, and order specification detail, directly impact LLM processing requirements and consequently affect overall system response characteristics.

Cross-domain performance analysis

The 13 test cases for “Create Manufacturing System” encompassed diverse manufacturing sectors to validate framework transferability across industrial domains:

- *Food processing (4 tests, $\mu = 280$ s, $\sigma = 25$ s)*: Bakery systems producing bread/sandwiches/toast, pandoro/panettone manufacturing with 13 specialized resources, beverage production lines, and pasta manufacturing facilities.
- *Energy and industrial equipment (3 tests, $\mu = 380$ s, $\sigma = 45$ s)*: Turbine manufacturing for oil & gas applications (17 workstations for 1MW capacity), power generation components, and renewable energy equipment assembly.
- *Automotive components (3 tests, $\mu = 320$ s, $\sigma = 35$ s)*: Engine component manufacturing, brake system production lines, and automotive electronics assembly.
- *General manufacturing (3 tests, $\mu = 300$ s, $\sigma = 30$ s)*: Mechanical component production, precision tooling manufacturing, and industrial machinery assembly.

Performance scaling correlates predictably with domain complexity: food processing demonstrates optimal efficiency due to linear production sequences, while energy/industrial equipment requires extended processing for sophisticated component interactions and quality requirements. Notably, the system exhibited greater difficulty in knowledge extraction when user requests contained incomplete information specifications. In scenarios where users provided vague or underspecified inputs, particularly in specialized domains like turbine manufacturing, the pre-trained LLM struggled to generate domain-specific technical parameters with the same accuracy as when detailed specifications were provided. This suggests that for highly specialized manufacturing contexts with incomplete user inputs, domain-specific model fine-tuning may be necessary to enhance parameter generation accuracy. While a generic LLM can generate reasonable general manufacturing processes, it lacks the specialized expertise to produce detailed

Bills of Process (BOP) for complex industrial equipment without adequate domain-specific information from users.

Baseline comparison with traditional ASMG

To provide empirical evidence of accessibility improvements, we conducted controlled comparison studies between our LLM-enhanced framework and a traditional data-driven ASMG baseline. The baseline system employed the identical simulation architecture and model construction components (Input, Workstation, and System classes) described in “[Step 3: simulation model construction](#)” section, but without the LLM-driven knowledge extraction interface.

The traditional approach required users to manually populate Excel spreadsheets containing structured templates for:

- Resource specifications (names, working/waiting consumption values, capacity, shifts)
- Product definitions with complete bill of processes (operation sequences, processing times, workstation assignments)
- Production orders (quantities, priorities, delivery dates)

Testing with non-expert users revealed significant accessibility barriers in the traditional approach. Users required an average of 35 min to complete system creation through Excel template compilation, with frequent assistance needed for understanding data structures and parameter relationships. Several models generated errors due to incorrect data entry, including mismatched resource-operation assignments, unrealistic timing specifications, and missing system dependencies.

In contrast, the LLM-enhanced framework enabled the same users to generate functionally equivalent manufacturing systems in 5–8 min through conversational requests without requiring technical assistance or prior training.

This comparison demonstrates a reduction in setup time, elimination of training requirements. Importantly, this baseline represents a comparison between ASMG methodologies. Traditional manual simulation development using conventional software would require days or weeks for equivalent model complexity, including time for software training and manual model construction.

Discussion

This section examines how our natural language-driven simulation methodology addresses the research questions from “[Introduction](#)” section. The practical application (“[Practical application](#)” section) demonstrates RQ1 by showing how

natural language interfaces enable non-expert users to create functional simulation models through conversational interaction. Performance testing (“[Performance testing](#)” section) addresses RQ2, revealing response times of 3–6 s for simple operations, 8–24 s for data management, and approximately 5 min for complex system creation. Cross-domain validation across food processing, energy equipment, automotive components, and general manufacturing demonstrates framework transferability, while comparison with Excel-based ASMG approaches shows measurable setup time reductions.

Our methodology demonstrates that LLM-ASMG integration can help address the core accessibility limitations constraining simulation adoption (Ghobakhloo & Ching, 2019; Hill, 2022). The four-phase framework enables direct transformation of conversational requests into executable simulation models, eliminating technical barriers identified in “[State-of-the-art overview](#)” section. The Four-Step Knowledge Extraction Pipeline addresses common LLM limitations through structured prompt engineering, while object-oriented simulation architecture ensures computational reliability. Performance results validate practical viability with response times representing substantial improvement over traditional development cycles requiring months of specialized effort (Hill, 2022). Cross-domain testing demonstrates transferability while empirical comparison shows measurable improvements in setup time and training requirements.

This transformation helps address the core accessibility challenge that has historically limited simulation technology to organizations with specialized resources and expertise (Ayandibu & Houghton, 2017; Ghobakhloo & Ching, 2019). Where conventional approaches require months of expert involvement, our framework generates functional models in minutes to hours across diverse manufacturing domains.

The following discussion examines three key aspects of our contribution. In “[Bridging the accessibility gap through natural language-driven simulation development](#)” section analyzes how performance metrics demonstrate practical viability in industrial contexts, showing that response times from 8 s to 5 min represent acceptable latency for production planning applications. In [Key advances for production planning in manufacturing systems](#)” section explores the key advances for manufacturing systems, including democratization of simulation technology and enhanced model development efficiency. In [Limitations and future research directions](#)” section discusses current limitations and identifies promising future research directions, including

domain-specific model enhancement and bidirectional communication capabilities.

Bridging the accessibility gap through natural language-driven simulation development

The performance metrics obtained from our testing provide evidence for the framework’s practical viability in industrial contexts. Response times ranging from 8 s for simple operations to 5 min for complete system generation represent acceptable latency for interactive production planning applications, where decision-making typically occurs over hours or days rather than requiring real-time responses. The observed scalability patterns, where processing time correlates directly with operational complexity, demonstrate predictable performance behavior that enables effective capacity planning and user expectation management, contrasting with the unpredictable timelines often associated with traditional simulation development projects.

The primary contribution lies in the integration of Large Language Models with Automatic Simulation Model Generation through a four-phase methodology that connects natural language understanding with executable simulation construction, addressing the challenge of making simulation technology more accessible while maintaining technical requirements for industrial decision support. Our approach presents a comprehensive end-to-end framework that transforms conversational user requests into functional discrete-event simulation models, eliminating the need for specialized programming knowledge or formal modeling expertise. An important innovation is the four-step knowledge extraction pipeline, which helps address common LLM limitations, including hallucinations, inconsistencies, and parameter generation errors, by combining action detection with context collection, template selection with prompt construction, parameter generation with industrial domain constraints, and validation with standardization processes. By incorporating industrial domain instructions as knowledge guardrails, the system guides LLM outputs toward realistic manufacturing parameters while enabling flexible interpretation of diverse natural language expressions. The object-oriented simulation framework introduces a three-class architecture comprising Input, Workstation, and System classes that enables automatic model reconfiguration and dynamic resource instantiation, supporting computational accuracy through validated parameter constraints and established discrete-event simulation principles while facilitating adaptive process mapping, scalable system configurations, and integration of diverse manufacturing scenarios without manual model reconstruction. The template-driven

architecture helps achieve reliable LLM performance by providing structured JSON schema specifications that align with simulation model requirements, ensuring consistent parameter extraction regardless of linguistic variations in user requests, allowing the use of general-purpose language models without specialized training while maintaining extensibility to new simulation domains, building upon traditional natural language processing applications in manufacturing by introducing simulation model generation capabilities through conversational interfaces that preserve analytical capabilities while expanding accessibility.

Key advances for production planning in manufacturing systems

The natural language-driven simulation methodology presented in this paper builds upon existing ASMG approaches (Cimino et al., 2025; Hughes et al., 2013; Reinhardt et al., 2019; Schlecht et al., 2023) while introducing natural language capabilities that substantially expand their accessibility. Key advances for intelligent manufacturing systems include:

- *Towards the democratization of simulation technology* by abstracting technical complexities behind natural language interfaces, this approach significantly lowers entry barriers, particularly for SMEs lacking specialized expertise (Ghobakhloo & Ching, 2019). Recent advances in LLM-enhanced manufacturing frameworks further support this trend toward accessible design tools (Mata et al., 2025). Users can leverage simulation capabilities without extensive technical training, potentially transforming decision-making from intuition-based to data-driven processes that align with Industry 4.0 principles.
- *Enhanced model development efficiency* traditional simulation projects typically require extensive manual specification and programming by specialized experts (Hill, 2022). Our methodology streamlines this process by automating the translation of natural language into simulation objects, enabling more responsive decision support in dynamic manufacturing environments and supporting agile production planning methodologies.
- *Improved human-machine interaction* the framework enables rapid model modification through natural language commands, allowing stakeholders to quickly test alternative configurations and explore broader solution spaces. This conversational approach to simulation control represents an advancement in human-centered manufacturing systems, facilitating more intuitive and robust planning processes.

Our methodology demonstrates adaptability across diverse production planning scenarios within manufacturing systems, including different manufacturing processes, product complexity levels, and resource configurations. The framework's current design is specifically optimized for production planning applications within manufacturing contexts. While domains such as logistics network design or predictive maintenance modeling would require different specialized approaches, our focused positioning within manufacturing production planning enables the framework to provide significant value and flexibility for this important industrial context.

Limitations and future research directions

This research represents an initial step toward natural language-driven simulation model generation, successfully addressing the gap in natural language interfaces (NLIs) highlighted by Behrendt et al. (2025) and Cimino et al. (2025), and demonstrating practical benefits in production planning contexts. While the demonstrated system successfully generates functional simulation models from natural language descriptions, some considerations must be acknowledged for future development. The current implementation effectively handles production systems with defined resources, products, and processes but focuses primarily on basic production planning scenarios including material flow, queue management, resource utilization, and basic scheduling. Our empirical testing revealed that the system exhibits reduced accuracy when processing incomplete or underspecified user inputs, particularly in highly specialized manufacturing domains. The general-purpose LLM demonstrates limitations in generating domain-specific technical parameters for complex industrial equipment when users provide vague specifications. This constraint becomes particularly evident in specialized contexts such as turbine manufacturing or precision engineering, where accurate Bills of Process (BOP) generation requires deep domain expertise that generic language models lack. The template-based approach that enables consistent interpretation introduces some structural constraints, meaning that highly novel system configurations may require template extensions. Additionally, the system's knowledge is inherently general rather than specific to particular industrial environments, and rapid model generation creates new considerations for validation approaches. Several promising directions for future research emerge from the current implementation:

- *Extending ASMG model complexity* developing more sophisticated simulation models to capture advanced manufacturing behaviors beyond basic production

planning, including material handling systems, complex scheduling algorithms, quality control processes, and physics-based interactions to enable higher-fidelity industrial system representations.

- *LLM fine-tuning and RAG integration for specialized output* training LLMs specifically for simulation data generation using manufacturing datasets and implementing RAG architectures to produce more precise technical specifications and industry-standard parameters. This approach combines domain-specific fine-tuning with real-time retrieval of relevant technical information from manufacturing knowledge bases during inference, reducing hallucinations and improving parameter accuracy by grounding LLM outputs in verified industrial data, thereby reducing the gap between generated values and real-world industrial benchmarks.
- *Bidirectional communication enhancement* implementing interactive dialogue capabilities between the LLM and users to facilitate iterative system development, enabling the system to ask clarifying questions, request additional specifications, and guide users through complex system creation processes. This includes integrating Reasoning Language Models (RLMs) that can perform multi-step logical reasoning to better understand incomplete specifications, identify potential conflicts in user requirements, and provide structured problem-solving approaches for complex manufacturing system design challenges. This becomes particularly important for managing incomplete user inputs by enabling the system to identify missing critical information and guide users toward providing necessary specifications.
- *Multi-modal interaction* extending the interface to support multi-modal inputs, including natural language, sketches, and diagrams, to enhance expressiveness and accessibility.
- *Enhanced validation and transparency* developing automated validation methods and explanation capabilities that can verify the logical consistency of automatically generated parameters while providing users with insight into the reasoning process behind parameter derivation.

As simulation systems in manufacturing continue to evolve toward more adaptive, data-driven architectures (Cimino et al., 2025; Steinbacher et al., 2023a), the integration with natural language interfaces could help improve accessibility while maintaining the analytical power needed for complex industrial decision-making. Our methodology demonstrates that this integration is possible and it can deliver practical benefits in production planning contexts, potentially transforming how organizations across the industrial spectrum approach simulation-based decision support.

Conclusion

This paper has presented a novel methodology integrating LLMs with Automatic Simulation Model Generation (ASMG) to address persistent barriers in simulation adoption across industrial domains, with particular focus on manufacturing production planning applications. Our framework transforms natural language requests into executable simulation models through a four-phase process: User Intent Expression, Knowledge Extraction using LLM-guided templates, Simulation Model Construction via object-oriented data-driven components employing a structured three-class architecture (Input, Workstation, and System), and Simulation Execution with comprehensive results analysis. The demonstrated implementation in manufacturing contexts shows how this approach enables users to create and analyze simulation models using everyday language, reducing development time while eliminating specialized programming requirements. Even when users provide over-specified and disorganized information, the system effectively extracts relevant parameters and generates reasonable manufacturing parameters based on domain knowledge. By abstracting technical complexities behind natural language interfaces, our methodology makes simulation accessible to a broader range of industrial stakeholders, particularly benefiting small and medium enterprises with limited resources. Our approach presents two core innovations: the template-based structure for guiding LLM outputs, which ensures consistent interpretation of user intent while constraining responses to mitigate hallucinations, and the data-driven ASMG implementation using discrete-event simulation principles, which enables automatic translation of these templates into executable simulation models. Together, these innovations bridge the gap between natural language understanding and simulation construction. While the current implementation effectively handles structured manufacturing environments focused on basic production planning, several opportunities emerge for expanding the methodology's capabilities. Future research directions include extending model complexity to capture more sophisticated industrial behaviors, fine-tuning LLMs for simulation data generation using manufacturing datasets, implementing bidirectional communication capabilities between users and the system, and developing multi-modal interaction interfaces. This natural language-driven simulation methodology offers an approach that reduces expertise, time, and financial barriers to simulation adoption while preserving the analytical capabilities needed for effective industrial decision support. As this approach evolves, it has the potential to expand the use of simulation technology across industrial sectors, enhancing operational efficiency and competitiveness in increasingly dynamic environments.

Appendix 1: Knowledge extraction algorithm

```

1: function EXTRACTKNOWLEDGE(userRequest, systemState)    ▷ Step 1: Action
   Detection & Context Collection
2:   actionType ← DetectActionType(userRequest)
3:   context ← CollectCurrentSystemState(systemState)
               ▷ Step 2: Template Selection & Prompt Construction
4:   template ← SelectJSONTemplate(actionType)
5:   prompt ← CombineRequestContextTemplate(userRequest, context, template)
6:   prompt ← AddDomainConstraints(prompt)
               ▷ Step 3: LLM Parameter Generation
7:   jsonResponse ← SendStructuredPromptToLLM(prompt)
               ▷ Step 4: Validation & Standardization
8:   attempts ← 0
9:   while attempts < 3 AND NOT IsValid(jsonResponse) do
10:    attempts ← attempts + 1
11:    prompt ← PrepareForRetry(prompt)
12:    jsonResponse ← SendStructuredPromptToLLM(prompt)
13:  end while
14:  if IsValid(jsonResponse) then
15:    return StandardizeFormat(jsonResponse)
16:  else
17:    return ReturnError()
18:  end if
19: end function
20: function DETECTACTIONTYPE(request)    ▷ Map natural language to action
   types using keywords
21:   for each actionType in actionMapping do
22:     if request contains keywords for actionType then
23:       return actionType
24:     end if
25:   end for
26:   return null
27: end function

```

Algorithm 1 Knowledge Extraction from Natural Language

Appendix 2: LLM-generated JSON specifications

This appendix contains the complete JSON specifications generated by the Large Language Model during the knowledge extraction phase.

Appendix 2.1: Resource specifications

```
{
  "action": "add_resources",
  "confidence": 0.9,
  "parameters": {
    "resources": [
      {
        "resourceName": "BandSaw",
        "workingConsumption": 5.2,
        "waitingConsumption": 0.5,
        "capacity": 3,
        "shiftsNumber": 2
      },
      {
        "resourceName": "DrillPress",
        "workingConsumption": 4.8,
        "waitingConsumption": 0.6,
        "capacity": 4,
        "shiftsNumber": 3
      }
    ]
  }
}
```

Listing 1 Resource creation JSON generated by LLM**Appendix 2.2: Product specifications**

```
{
  "action": "add_product",
  "confidence": 0.9,
  "parameters": {
    "name": "WoodenTable",
    "priority": 2,
    "active": "Yes",
    "operations": [
      {
        "Operation": "CuttingLegs",
        "Pre-processing Time": 0.5,
        "Processing Time": 30.0,
        "Post-processing Time": 0.5,
        "Workstation": "WoodSaw"
      }
    ]
  }
}

{
  "action": "add_product",
  "confidence": 0.9,
  "parameters": {
    "name": "WoodenChair",
    "priority": 2,
    "active": "Yes",
    "operations": [
      {
        "Operation": "CuttingSeat",
        "Pre-processing Time": 0.5,
        "Processing Time": 1.0,
        "Post-processing Time": 0.5,
        "Workstation": "WoodSaw"
      }
    ]
  }
}
```

Listing 2 Products specification JSON generated by LLM

Appendix 2.3: Order specifications

```
{
  "action": "add_order",
  "confidence": 0.9,
  "parameters": {
    "Customer - Order Nr.": "ORD-001",
    "Priority": 2,
    "Product Type": "WoodenChair",
    "Quantity": 10,
    "Expected Production Start Date": "2025-03-21",
    "Expected Delivery Date": "2025-04-02"
  }
}

{
  "action": "add_order",
  "confidence": 0.9,
  "parameters": {
    "Customer - Order Nr.": "ORD-002",
    "Priority": 2,
    "Product Type": "WoodenTable",
    "Quantity": 15,
    "Expected Production Start Date": "2025-03-21",
    "Expected Delivery Date": "2025-04-04"
  }
}
```

Listing 3 Order specifications JSON generated by LLM

Appendix 3. Performance distribution analysis

See Fig. 7.

Performance Testing Results: Normal Distributions for All Action Types



Fig. 7 Statistical Distributions for all action types

Acknowledgements This work was supported by the “National Centre for HPC, Big Data and Quantum Computing (HPC)” (Project ID: CN_00000013, CUP: H23C22000360005)—funded under the National Recovery and Resilience Plan, Mission 4, Component 2, Investment 1.4 “Strengthening research structures and creation of national R&D champions on some Key Enabling Technologies” financed by the European Union—NextGenerationEU.

Funding Open access funding provided by Università della Calabria within the CRUI-CARE Agreement.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alieva, J., & Haartman, R. (2020). Digital MudA-the new form of waste by industry 4.0. *Operations and Supply Chain Management: An International Journal*, 13, 269–278.
- Almeida, E., & Tilbury, D. (2004). Automatic logic generation for reconfigurable cell-based manufacturing systems. In *IFAC proceedings volumes (IFAC-PapersOnline)* (vol. 37, pp. 33–38). [http://doi.org/10.1016/S1474-6670\(17\)30718-8](http://doi.org/10.1016/S1474-6670(17)30718-8)
- Ayandibu, A. O., & Houghton, J. (2017). The role of small and medium scale enterprise in local economic development (LED). *Journal of Business and Retail Management Research*, 11(2), 133–139.
- Badini, S., Regondi, S., Frontoni, E., & Pugliese, R. (2023). Assessing the capabilities of ChatGPT to improve additive manufacturing troubleshooting. *Advanced Industrial and Engineering Polymer Research*, 6, 278–287.
- Behrendt, S., Altenmüller, T., May, M. C., Kuhnle, A., & Lanza, G. (2025). Real-to-sim: Automatic simulation model generation for a digital twin in semiconductor manufacturing. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-025-02572-x>
- Billiet, H., & Stark, R. (2022). A framework for rescheduling a fixed-layout assembly system using discrete-event simulation. In *Proceedings—Winter simulation conference* (vol. 2022-Decem pp. 1762–1771). <https://doi.org/10.1109/WSC57314.2022.10015383>
- Burnett, G. A., Medeiros, D. J., Finke, D. A., & Traband, M. T. (2008). Automating the development of shipyard manufacturing models. In *Proceedings—Winter simulation conference* (pp. 1761–1767). <https://doi.org/10.1109/WSC.2008.4736264>
- Cadi, A. A. E., Gharbi, A., Dhoubi, K., & Artiba, A. (2021). Joint production and preventive maintenance controls for unreliable and imperfect manufacturing systems. *Journal of Manufacturing Systems*, 58, 263–279. <https://doi.org/10.1016/j.jmsy.2020.12.003>
- Castiglione, C. (2024). Automated generation of digital models for manufacturing systems: The event-centric process mining approach. *Computers and Industrial Engineering*, 197, 110596.
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., et al. (2024). A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3), 1–45.
- Cheng, D., Huang, S., Bi, J., Zhan, Y., Liu, J., Wang, Y., Sun, H., Wei, F., Deng, D., & Zhang, Q. (2023). *Uprise: Universal prompt retrieval for improving zero-shot evaluation*. arXiv preprint [arXiv:2303.08518](https://arxiv.org/abs/2303.08518)
- Cimino, A., Elbasheer, M., Longo, F., Mirabelli, G., Solina, V., & Veltri, P. (2025). Automatic simulation models generation in industrial systems: A systematic literature review and outlook towards simulation technology in the industry 5.0. *Journal of Manufacturing Systems*, 80, 859–882.
- Cimino, A., Longo, F., Mirabelli, G., Solina, V., & Veltri, P. (2024). Enhancing internal supply chain management in manufacturing through a simulation-based digital twin platform. *Computers and Industrial Engineering*, 198, 110670.
- Collins, A. J., Thaviphoke, Y., & Tako, A. A. (2023). Using strategic options development and analysis (SODA) to understand the simulation accessibility problem. *Journal of the Operational Research Society*, 74, 2143–2164. <https://doi.org/10.1080/01605682.2022.2129481>
- Du, J., He, Q., & Fan, X. (2013). Automating generation of the assembly line models in aircraft manufacturing simulation. In *Proceedings—2013 IEEE international symposium on assembly and manufacturing, ISAM 2013* (pp. 155–159). <https://doi.org/10.1109/ISAM.2013.6643515>
- Elbasheer, M., Longo, F., Mirabelli, G., & Solina, V. (2024). Flexible symbiosis for simulation optimization in production scheduling: A design strategy for adaptive decision support in industry 5.0. *Journal of Manufacturing and Materials Processing*, 8(6), 275.
- Fan, H., Liu, X., Fuh, J. Y. H., Lu, W. F., & Li, B. (2025). Embodied intelligence in manufacturing: Leveraging large language models for autonomous industrial robotics. *Journal of Intelligent Manufacturing*, 36(2), 1141–1157.
- Ferreira, W., Armellini, F., & Santa-Eulalia, L. A. (2020). Simulation in industry 4.0: A state-of-the-art review. *Computers and Industrial Engineering*, 149, 106868. <https://doi.org/10.1016/j.cie.2020.106868>
- Friederich, J., Francis, D. P., Lazarova-Molnar, S., & Mohamed, N. (2022). A framework for data-driven digital twins for smart manufacturing. *Computers in Industry*, 136, 103586. <https://doi.org/10.1016/j.compind.2021.103586>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey*. arXiv preprint [arXiv:2312.10997](https://arxiv.org/abs/2312.10997)
- Garcia, F. A., Eremeev, P., Devriendt, H., Naets, F., Metin, H., & Ozer, M. (2023). *Automatic generation of digital-twins in advanced manufacturing: A feasibility study*. <https://doi.org/10.1109/ICCA457653.2023.10152364>
- Geurtsen, M., Adan, I., & Atan, Z. (2023). Deep reinforcement learning for optimal planning of assembly line maintenance. *Journal of Manufacturing Systems*, 69, 170–188.
- Ghobakhloo, M., & Ching, N. T. (2019). Adoption of digital technologies of smart manufacturing in SMEs. *Journal of Industrial Information Integration*, 16, 100107. <https://doi.org/10.1016/j.jii.2019.100107>
- Göppert, A., Grahn, L., Rachner, J., Grunert, D., Hort, S., & Schmitt, R. H. (2023). Pipeline for ontology-based modeling and automated deployment of digital twins for planning and control of manufacturing systems. *Journal of Intelligent Manufacturing*, 120, 1–20.
- Hadi, M. U., Qureshi, R., Shah, A., Irfan, M., Zafar, A., Shaikh, M. B., Akhtar, N., Wu, J., Mirjalili, S., et al. (2023). Large language models: A comprehensive survey of its applications, challenges, limitations, and future prospects. *Authorea*, 1, 1–26.

- Harrison, J. R., Lin, Z., Carroll, G. R., & Carley, K. M. (2007). Simulation modeling in organizational and management research. *Academy of Management Review*, 32, 1229–1245.
- Hill, T. (2022). *Simulation is a window into the future of your manufacturing operation*. National Institute of Standards and Technologies. Verkkosivu. Saatavissa (viitattu 12.4. 2023). <https://www.nist.gov/blogs/manufacturing-innovation-blog/simulation-window-future-your-manufacturing-operation>
- Hughes, R. W. C., Scott, R., & Ridgway, K. (2013). Automatic simulation model generation for supporting facility planning in SMEs. In *Modelling and simulation 2013—European simulation and modelling conference, ESM 2013* (pp. 62–67).
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38.
- Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2024). A survey on large language models for code generation. arXiv preprint [arXiv:2406.00515](https://arxiv.org/abs/2406.00515)
- Kaiser, B., Reichle, A., & Verl, A. (2022). Model-based automatic generation of digital twin models for the simulation of reconfigurable manufacturing systems for timber construction. *Procedia CIRP*, 107, 387–392. <https://doi.org/10.1016/j.procir.2022.04.063>
- Katsaliaki, K., & Mustafee, N. (2019). Distributed simulation of supply chains in the industry 4.0 era: A state of the art field overview. In M. Gunal (Ed.), *Simulation for industry 4.0: Past, present, and future* (pp. 55–80). Springer.
- Krenczyk, D., Kempa, W. M., Kalinowski, K., Grabowik, C., & Paprocka, I. (2018). Integration of manufacturing operations management tools and discrete event simulation. *IOP Conference Series: Materials Science and Engineering*, 400, 022037. <https://doi.org/10.1088/1757-899X/400/2/022037>
- Lee, J., & Su, H. (2023). *A unified industrial large knowledge model framework in smart manufacturing*. arXiv preprint [arXiv:2312.14428](https://arxiv.org/abs/2312.14428)
- Leißau, M., & Laroque, C. (2024). Breaking barriers in semiconductor simulations: An automated low-code framework for model-structure synchronisation and large-scale simulation studies. In *2024 Winter simulation conference (WSC)* (pp. 1919–1930). IEEE.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Longo, F., Mirabelli, G., Padovano, A., & Solina, V. (2023). The digital supply chain twin paradigm for enhancing resilience and sustainability against COVID-like crises. *Procedia Computer Science*, 217, 1940–1947. <https://doi.org/10.1016/j.procs.2022.12.394>
- Lugaresi, G., & Matta, A. (2021). Automated digital twins generation for manufacturing systems: A case study. *IFAC-PapersOnLine*, 54, 749–754. <https://doi.org/10.1016/j.ifacol.2021.08.087>
- Maier, J. B., Colangelo, E., Hinrichsen, T.-F., Tran, D. K., Wiendahl, H.-H., & Huber, M. F. (2024). Simulation discovery and semi-automatic scenario generation for evaluation of turbulence in production systems. *Procedia CIRP*, 130, 1623–1631.
- Mata, O., Ponce, P., Perez, C., Ramirez, M., Anthony, B., Russel, B., Apte, P., MacCleery, B., & Molina, A. (2025). Digital twin designs with generative AI: Crafting a comprehensive framework for manufacturing systems. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-025-02583-8>
- Mo, F., Querejeta, M. U., Hellewell, J., Rehman, H. U., Rezabal, M. I., Chaplin, J. C., Sanderson, D., & Ratchev, S. (2023). Plc orchestration automation to enhance human–machine integration in adaptive manufacturing systems. *Journal of Manufacturing Systems*, 71, 172–187. <https://doi.org/10.1016/j.jmsy.2023.07.015>
- Mueller, R., Alexopoulos, C., & McGinnis, L. F. (2007). Automatic generation of simulation models for semiconductor manufacturing. In *Proceedings—Winter simulation conference* (pp. 648–657). <https://doi.org/10.1109/WSC.2007.4419658>
- Nasereddin, M., Mullens, M. A., & Cope, D. (2007). Automated simulator development: A strategy for modeling modular housing production. *Automation in Construction*, 16, 212–223. <https://doi.org/10.1016/j.autcon.2006.04.003>
- Negri, E., Pandhare, V., Cattaneo, L., Singh, J., Macchi, M., & Lee, J. (2021). Field-synchronized digital twin framework for production scheduling with uncertainty. *Journal of Intelligent Manufacturing*, 32(4), 1207–1228.
- Orlova, E. V. (2022). Design technology and AI-based decision making model for digital twin engineering. *Future Internet*, 14, 248. <https://doi.org/10.3390/fi14090248>
- Patil, R., & Gudivada, V. (2024). A review of current trends, techniques, and challenges in large language models (LLMs). *Applied Sciences*, 14(5), 2074.
- Reinhardt, H., Weber, M., & Putz, M. (2019). A survey on automatic model generation for material flow simulation in discrete manufacturing. *Procedia CIRP*, 81, 121–126. <https://doi.org/10.1016/j.procir.2019.03.022>
- Santos, C. H., Queiroz, J. A. D., Leal, F., & Montevechi, J. A. B. (2022). Use of simulation in the industry 4.0 context: Creation of a digital twin to optimise decision making on non-automated process. *Journal of Simulation*, 16, 284–297.
- Sarraj, R., Ballot, E., Pan, S., Hakimi, D., & Montreuil, B. (2014). Interconnected logistic networks and protocols: Simulation-based efficiency assessment. *International Journal of Production Research*, 52(11), 3185–3208.
- Schlecht, M., Guio, R., & Köbler, J. (2023). Automated generation of simulation model in context of industry 4.0. *International Journal of Modelling and Simulation*, 45, 441–453. <https://doi.org/10.1080/02286203.2023.2206075>
- Shi, H., Xu, Z., Wang, H., Qin, W., Wang, W., Wang, Y., Wang, Z., Ebrahimi, S., & Wang, H. (2024). Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*. <https://doi.org/10.1145/3735633>
- Si, C., Gan, Z., Yang, Z., Wang, S., Wang, J., Boyd-Graber, J., & Wang, L. (2022). *Prompting gpt-3 to be reliable*. arXiv preprint [arXiv:2210.09150](https://arxiv.org/abs/2210.09150)
- Soares, M., Ribeiro, A., Vasconcelos, T., Barros, M., Castro, C., Vilarinho, C., & Carvalho, J. (2023). Challenges of digital waste marketplace—the upvalue platform. *Sustainability*, 15, 11235. <https://doi.org/10.3390/su151411235>
- Sommer, M., Stjepandic, J., Stobrawa, S., & Soden, M. V. (2020). Automated generation of a digital twin of a manufacturing system by using scan and convolutional neural networks. *Advances in Transdisciplinary Engineering*, 12, 363–372. <https://doi.org/10.3233/ATDE200095>
- Son, Y. J., & Wysk, R. A. (2001). Automatic simulation model generation for simulation-based, real-time shop floor control. *Computers in Industry*, 45(3), 291–308.
- Son, Y. J., Wysk, R. A., & Jones, A. T. (2003). Simulation-based shop floor control: Formal model, model generation and control interface. *IIE Transactions (Institute of Industrial Engineers)*, 35, 29–48. <https://doi.org/10.1080/07408170304428>
- Soufi, Z., Mestiri, S., David, P., Yahouni, Z., & Fottner, J. (2024). A material handling system modeling framework: A data-driven approach for the generation of discrete-event simulation models. *Flexible Services and Manufacturing Journal*, 37, 1–30.
- Steinbacher, L. M., Düe, T., Veigt, M., & Freitag, M. (2023). Automatic model generation for material flow simulations of third-party logistics. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-023-02257-3>
- Steinbacher, L. M., Rippel, D., Schulze, P., Rohde, A.-K., & Freitag, M. (2023). Quality-based scheduling for a flexible job shop.

- Journal of Manufacturing Systems*, 70, 202–216. <https://doi.org/10.1016/j.jmsy.2023.07.005>
- Sturrock, D. T. (2019). Traditional simulation applications in industry 4.0. In M. Gunal (Ed.), *Simulation for industry 4.0* (Vol. 4, pp. 39–54). Springer. https://doi.org/10.1007/978-3-030-04137-3_3
- Tan, W. J., Seok, M. G., & Cai, W. (2023). *Automatic model generation and data assimilation framework for cyber-physical production systems* (pp. 73–83). <https://doi.org/10.1145/3573900.3591112>
- Tiago, Z., Costa, C. A., Righi, R., Lima, M., & Li, G. (2020). Predictive maintenance in the industry 4.0: A systematic literature review. *Computers and Industrial Engineering*, 150, 17. <https://doi.org/10.1016/j.cie.2020.106889>
- Tonmoy, S., Zaman, S., Jain, V., Rani, A., Rawte, V., Chadha, A., & Das, A. (2024). *A comprehensive survey of hallucination mitigation techniques in large language models*. arXiv preprint [arXiv:2401.01313](https://arxiv.org/abs/2401.01313)
- Uhlmann, I. R., & Frazzon, E. M. (2018). Production rescheduling review: Opportunities for industrial integration and practical applications. *Journal of Manufacturing Systems*, 49, 186–193.
- Wang, J., Chang, Q., Xiao, G., Wang, N., & Li, S. (2011). Data driven production modeling and simulation of complex automobile general assembly plant. *Computers in Industry*, 62, 765–775. <https://doi.org/10.1016/j.compind.2011.05.004>
- Wang, T., Fan, J., & Zheng, P. (2024). An LLM-based vision and language cobot navigation approach for human-centric smart manufacturing. *Journal of Manufacturing Systems*, 75, 299–305.
- Wang, Z., Ezukwoke, K., Hoayek, A., Batton-Hubert, M., & Boucher, X. (2025). Natural language processing (NLP) and association rules (AR)-based knowledge extraction for intelligent fault analysis: A case study in semiconductor industry. *Journal of Intelligent Manufacturing*, 36(1), 357–372.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). *A prompt pattern catalog to enhance prompt engineering with ChatGPT*. arXiv preprint [arXiv:2302.11382](https://arxiv.org/abs/2302.11382)
- Winkels, J., Özkul, F., Sutherland, R., Löhn, J., Wenzel, S., & Rehof, J. (2024). Component-based synthesis of structural variants of simulation models for changeable material flow systems. In *2024 Winter simulation conference (WSC)* (pp. 1657–1668). IEEE.
- Wocker, M., Ostermeier, F., Wanninger, T., Zwinkau, R., & Deuse, J. (2023). Flexible job shop scheduling with preventive maintenance consideration. *Journal of Intelligent Manufacturing*, 35, 1517–1539. <https://doi.org/10.1007/s10845-023-02114-3>
- Wy, J., Jeong, S., Kim, B.-I., Park, J., Shin, J., Yoon, H., & Lee, S. (2011). A data-driven generic simulation model for logistics-embedded assembly manufacturing lines. *Computers and Industrial Engineering*, 60, 138–147. <https://doi.org/10.1016/j.cie.2010.10.011>
- Xie, K., Zhang, L., Wang, X., Wang, K., & Li, Y. (2024). Integrated modelling and simulation method of hybrid systems based on x language. *IET Collaborative Intelligent Manufacturing*, 6(4), 70006.
- Zhang, L., Zhang, Y., Niyato, D., Ren, L., Gu, P., Chen, Z., Laili, Y., Cai, W., & Bruzzone, A. (2025). GenAI for simulation model in model-based systems engineering. arXiv preprint [arXiv:2503.06422](https://arxiv.org/abs/2503.06422)
- Zhang, L., Zhou, L., Ren, L., & Laili, Y. (2019). Modeling and simulation in intelligent manufacturing. *Computers in Industry*, 112, 103123. <https://doi.org/10.1016/j.compind.2019.08.004>
- Zonta, T., Costa, C. A. D., Rosa Righi, R., Lima, M. J., Trindade, E. S., & Li, G. P. (2020). Predictive maintenance in the industry 4.0: A systematic literature review. *Computers and Industrial Engineering*, 150, 106889.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.